

SZAKDOLGOZAT

Páll László

2025



NYÍREGYHÁZI
EGYETEM

**IoT alapú egészségügyi monitoring rendszer fejlesztése
krónikus betegek számára**

Páll László

Programtervező informatikus

Konzulens: Vegera József

20...

NYILATKOZAT

Alulírott

Név: Pál László

Szak: Programtervező Informatikus

NEPTUN kód: LX1742

jelen nyilatkozat aláírásával kijelentem, hogy a

IoT alapú egészségügyi monitoring rendszer fejlesztése
krónikus betegek számára

című szakdolgozat önálló munkám eredménye, saját szellemi termékem, abban a hivatkozások és idézések standard szabályait következetesen alkalmaztam, mások által írt részeket a megfelelő idézés nélkül nem használtam fel. A dolgozat készítése során betartottam a szerzői jogról szóló 1999. évi LXXVI. törvény szabályait, valamint a Nyíregyházi Egyetem által előírt, a szakdolgozat készítésére vonatkozó szabályokat. Plágium semmilyen formában nem megengedett, hivatkozás nélkül a saját alkotások sem idézhetők (önplágium tilalma). A mesterséges intelligencia használata csak transzparens és etikus módon engedélyezett. Kijelentem továbbá, hogy sem a dolgozatot, sem annak bármely részét nem nyújtottam be szakdolgozatként.

Nyíregyháza, április 28.

Pál László

a hallgató aláírása

Tartalom

1. Bevezetés	6
1.1 A téma aktualitása	6
1.2 Személyes motiváció	6
1.3 A kutatás célja és a problémafelvetés.....	7
1.4 Hipotézisek és módszertan.....	8
2. Az IoT alapú egészségügyi monitoring fogalma, történeti fejlődése, formái és módszertani kérdései.....	9
2.1 A krónikus betegellátás kihívásai és az IoT-alapú megoldások	9
2.1.1 A jelenlegi rendszer hiányosságai.....	10
2.1.2 Az IoT technológia előnyei.....	10
2.1.3 Az otthoni monitorozás problémái.....	12
2.1.4 Társadalmi és földrajzi akadályok.....	12
2.2 IoT-technológia az egészségügyben	14
2.3 Viselhető szenzorok és kommunikációs protokollok	14
2.4 Módszertani kihívások az adatkezelésben.....	16
2.5 Rendszertípusok összehasonlítása.....	17
3. Piaci Körkép, biztonsági kihívások és jogi szabályozás	19
3.1 Jogi háttér (GDPR és egészségügyi adatvédelem)	21
4. Rendszer tervezése	22
4.1 Funkcionális követelmények	23
4.1.1 Felhasználói szerepkörök és használati esetek (Use Cases)	23
4.1.2 Adatfolyam és folyamatleírás	24
4.2 Nem funkcionális követelmények.....	25
4.3 Adatbázis-tervezés	26
4.3.1 Adattáblák definíciója és szerkezete	26
5. Fejezet A rendszer architekturális felépítése és implementációja	29
5.1. Felhasználói szerepkörök és jogosultságkezelés	31
5.2. Vizualizáció: A Páciens Műszerfal (Dashboard).....	32
5.3. A Riasztási Modul (Alarm Logic) működése.....	33
6. A rendszer megvalósítása és technológiai háttere	34
6.1. IntelliJ IDEA és JDK.....	34
6.1.2. Build eszköz és függőségkezelés (Maven)	35
6.1.3. Verziókezelés és IoT szimuláció.....	35
6.2. A választott keretrendszer: Spring Boot.....	36
6.2.1. Dependency Injection (Függőséginjektálás).....	36
6.2.2. Adatbázis réteg (JPA és Hibernate)	36

6.3. Az üzleti logika implementációja (Service Layer)	38
6.3.1. Adatvalidáció	38
6.3.2. Dinamikus határérték-figyelés	38
6.4. API és Vezérlők (Controllers)	39
6.4.1. MQTT integráció (MQTTconfig és MQTTListener)	39
6.4.2. Bootstrap CSS.....	40
7. A prototípus bemutatása és tesztelése	41
7.1 Az MQTT protokoll szerepe a rendszerben	41
7.2 Normál működés tesztelése	43
7.3 Riasztási esemény szimulációja	44
7.4 A tesztelés korlátai és értékelése	45
8. Összegzés és kitekintés	48
8.1 Az elért eredmények összefoglalása	48
8.2 A hipotézisek kiértékelése	49
8.3 A prototípus korlátai	50
8.4 Jövőbeli fejlesztési lehetőségek	51
Irodalomjegyzék	52
SZAKDOLGOZATI BÍRÁLÓLAP	56

1. Bevezetés

1.1 A téma aktualitása

A 21. században a technológiai fejlődés, különösen az Internet of Things (IoT) rohamos terjedése, alapjaiban megváltoztatja az egészségügyi ellátórendszereket. Az okoseszközök és viselhető szenzorok elterjedése forradalmasítja a betegellátást, lehetővé téve a páciensek egészségügyi állapotának folyamatos, valós idejű monitorozását otthoni környezetben is. Ez a technológiai váltás különösen a krónikus betegségekkel élők számára nyújt új lehetőségeket, akiknek állapota folyamatos felügyeletet igényel. **(Abdulkadhim és Hasan, 2023)** *(Az IoT definíciójára és elterjedésére vonatkozóan).*

A krónikus megbetegedések, mint például a szív- és érrendszeri betegségek, a cukorbetegség vagy a krónikus obstruktív tüdőbetegség (COPD), világszerte és Magyarországon is a vezető halálokok közé tartoznak, és jelentős terhet jelentenek az egészségügyi ellátórendszerre. Ezen betegségek menedzselése gyakori orvosi ellenőrzést, a páciens aktív közreműködését és az életmódbeli előírások szigorú betartását teszi szükségessé. A hagyományos ellátási modellek azonban gyakran elmaradottak, és csak a tünetek súlyosbodásakor tesznek lehetővé beavatkozást. **(BIJAM, 2023)(e-Health Connect, 2022).** *(A krónikus betegségek kezelésének nehézségeire).*

Az IoT-alapú egészségügyi monitoring rendszerek proaktív megközelítést kínálnak. Ezek a rendszerek képesek folyamatosan gyűjteni, elemezni a létfontosságú paramétereket, mint a vérnyomás, pulzusszám, vércukorszint, a testoxigén-szaturáció (SpO2) és a testhőmérséklet. Az adatok automatizálása, továbbítása és alapszintű elemzése lehetővé teszi az egészségügyi szakemberek számára, hogy távolról is figyelemmel kísérjék pácienseik állapotát, és időben észleljék a kellemetlen változásokat, megelőzve ezzel a súlyosabb szövődmények kialakulását. **(e-Health Connect, 2022).** *(Az IoT proaktív szerepére).*

1.2 Személyes motiváció

A témaválasztásomat a személyes érintettség és a szakmai érdeklődés kettőssége motiválta. Mivel szülőm az egészségügyben dolgozik, napi szintű, közvetlen betekintést nyerhetek az ellátórendszer kihívásaiba. Rendszeresen szembesülök azokkal a nehézségekkel, amelyeket a sokszor elavult, nem digitális alapú folyamatok okoznak a betegellátásban, különösen a krónikus betegek hosszú távú gondozása során. Ez a közvetlen tapasztalat ébresztett rá arra, hogy az informatikai és mérnöki tudásommal hogyan lehetne hatékonyan hozzájárulni ezen problémák

enyhítéséhez. Személyes és szakmai érdeklődésem egyaránt a beágyazott rendszerek és az adat-elemzés területeire irányul, melyek ötvöződnek egy IoT-alapú egészségügyi monitoring rendszer fejlesztésében.

1.3 A kutatás célja és a problémafelvetés

Ez a kettős egyszerre személyes és szakmai motiváció meghatározó szerepet játszott abban, hogy a témaválasztás ne csupán egy elvont informatikai feladat megoldására irányuljon, hanem valódi társadalmi szükségletet kielégítő rendszer tervezésére. Az egészségügy digitális átalakulása ugyanis nem kizárólag technológiai kérdés: magában foglalja a beteg és az orvos kapcsolatának újraértelmezését, az ellátás szervezésének átalakítását, valamint az adatvédelmi és etikai normák következetes érvényesítését is. Mindezek megértéséhez elengedhetetlen volt az a közvetlen tapasztalat, amelyet a szerző az egészségügyi munkakörnyezetből merített.

A krónikus betegségek hosszú távú kezelése számos olyan adminisztratív és kommunikációs problémát vet fel, amelyek megoldására a hagyományos egészségügyi rendszerek nem mindig képesek. A betegek állapotának figyelemmel kísérése az esetek jelentős részében papíralapú dokumentáción és ritka személyes konzultációkon alapul, ami időbeli késedelmet okoz a klinikai döntéshozatalban. Az informatika területén szerzett ismeretek alkalmazása ezért nem pusztán akadémiai vállalkozás, hanem közvetlen gyakorlati relevanciával bíró tevékenység, amely érdemben hozzájárulhat a betegellátás hatékonyságának és minőségének javításához.

A szakdolgozat elkészítése során ezért nem csupán egy elméleti problémát kívánok megoldani, hanem egy olyan, rendszert a gyakorlatban is potenciálisan alkalmazható lenne. A szakdolgozat célja egy olyan IoT-alapú egészségügyi monitoring rendszer koncepciójának kidolgozása, amely krónikus betegségben szenvedő páciensek számára lehetővé teszi a kulcsfontosságú egészségügyi paraméterek folyamatos, otthoni környezetben történő mérését, az adatok biztonságos továbbítását egy központi szerverre.

A központi probléma, amelyre a dolgozat megoldást keres: hogyan lehet egy megbízható, felhasználóbarát és költséghatékony rendszert létrehozni, amely a modern szenzortechnológiát és a hálózati kommunikációt felhasználva hatékony támogatást nyújt a krónikus betegek távoli menedzselésében?

A kutatás a rendszer teljes működési folyamatát felöleli, kezdve a releváns élettani paraméterek (vérnyomás, pulzus, SpO2 stb.) mérésére alkalmas egészségügyi szenzorok és eszközök elemzésével. Ezt követően maga a vizsgálat és felhasználható adatok feldolgozására, valamint alapszintű elemzésére fókuszál, amelynek célja a kritikus állapotok és a negatív tendenciák

automatizált felismerésére szolgál. A folyamat zárásaként a dolgozat a feldolgozott információk hatékony vizualizációjával és a páciensek, valamint az egészségügyi szakemberek számára tervezett, felhasználóbarát felületekkel foglalkozik.

A kutatás és a gyakorlati megvalósítás során generatív MI-alapú segédeszköz is alkalmazásra került a kódfejlesztési, hibakeresési, irodalomkeresési irányok meghatározásához. Ugyanakkor a dolgozatban szereplő műszaki megoldások, mérések, eredmények és következtetések minden esetben saját ellenőrzés és önálló validáció alapján kerültek kialakításra.

1.4 Hipotézisek és módszertan

A szakdolgozat legfőbb hipotézise maga az elavult kórházi eszközök problémája, amik gyakran hamis információkat adnak vagy nincsenek működési állapotban állapotba. Egy alapszintű adatelemzési funkció implementálásával, ami képes proaktívan jelezni állapotromlást, ezáltal potenciálisan csökkentve a sürgősségi ellátás szükségességét. Ezen kívül a beteg fontos élettani paramétereinek megbízható és folyamatos monitorozási adatok automatizált továbbítása, amely lehetővé tenné az orvosok számára a páciensek állapotának hatékonyabb távoli megvizsgálását.

A kutatás során szakirodalmi elemzésre, rendszerszintű összehasonlításra és módszertani értékelésre fog sor kerülni. A dolgozat a hazai és nemzetközi szakirodalmat dolgozza fel az IoT-alapú egészségügyi rendszerek előnyei, hátrányai és korlátait rátekintve az adatvédelemre, adatbiztonságra is. Az elemzés rátér a különböző architektúrákra, kommunikációs protokollok és adatkezelési megoldásokkal összevetésével tárja fel a fejlesztési lehetőségeket.

Maga a gondolatmenet végig megy a problémafelvetéstől a megoldási javaslatig és annak értékeléséig. A bevezetést követően a kapcsolódó szakirodalmi és technológiai háttérrel dolgozza fel, bemutatva egészségügyi monitorozás, az IoT-architektúrák, a kommunikációs technológiákat és az adatbiztonság alapjait részletes kitérve azokra. Majd ezután ismerteti a kutatási problémát és a fejlesztéssel kapcsolatos célkitűzéseket, valamint módszertanát, elemzéseinek lépéseit. Mindezek után a kutatások eredményeit, valamint felvázolását fejteném ki a krónikus betegek számára az egészségügyi monitoring rendszerek alkalmazási lehetőségeit, működési eleveit, valamint megvalósítási elméleteket. Végül összegzem a kutatás főbb megállapításait, valamint a jövőbeli fejlesztéseket és kutatási irányokat.

A kutatási hipotézisek megfogalmazása során figyelembevételre kerültek azok a nemzetközi szakirodalmi eredmények is, amelyek alátámasztják az IoT-alapú monitorozás klinikai hasznosságát. Több vizsgálat kimutatta, hogy a folyamatos, automatizált élettani paraméter-gyűjtés szignifikánsan csökkenti a nem tervezett kórházi újrafelvételek számát, különösen

szívelégtelenségben és krónikus obstruktív tüdőbetegségben szenvedő páciensek esetén. Ezek az eredmények alátámasztják azt az alapfeltevést, miszerint a proaktív, adatvezérelt ellátási modell, amelynek megvalósítására a jelen szakdolgozat törekszik valós egészségügyi és gazdasági előnyöket hordoz magában.

A módszertani keretrendszer kialakításakor a szerző tudatosan törekedett arra, hogy az elméleti megalapozás és a gyakorlati implementáció egyensúlyban legyen. A szakirodalmi elemzés nem öncélú összefoglalás, hanem közvetlen kapcsolódási pontokat teremt a rendszertervezési döntésekhez: az azonosított hiányosságok és bevált megközelítések egyaránt beépülnek a fejlesztési folyamatba. Ez a módszertani következetesség biztosítja, hogy az elkészült prototípus ne csupán technológiai demonstráció legyen, hanem egy jól indokolt, szakmailag megalapozott szoftvermegoldás, amelynek minden tervezési döntése visszavezethető az irodalomban azonosított szükségletekre és elvárásokra.

A dolgozat szerkezete tudatosan tükrözi az alkalmazott kutatómódszertani elveket. A bevezető fejezet szerepe kettős: egyrészt kontextust teremt a téma megértéséhez, másrészt rávilágít azokra a valódi problémákra, amelyek megoldására a fejlesztés irányul. A problémafelvetés konkrétsága a hazai egészségügyi ellátórendszer digitális lemaradása, az otthoni monitorozás elégtelensége, a krónikus betegek reaktív ellátási modellje megalapozza az ezt követő elméleti és gyakorlati fejezetek szükségességét. A bevezetés így nem csupán formai elem, hanem a teljes dolgozat argumentációs láncolatának kiindulópontja.

2. Az IoT alapú egészségügyi monitoring fogalma, történeti fejlődése, formái és módszertani kérdései

2.1 A krónikus betegellátás kihívásai és az IoT-alapú megoldások

A krónikus betegségek ellátása világszerte, így Magyarországon is, jelentős kihívást jelent az egészségügyi rendszerek számára. A hosszú távú gondozást igénylő páciensek rendszeres állapotellenőrzést, gyógyszeres terápiát és életmódbeli előírásokat követelnek meg, miközben az ellátórendszerek gyakran túlterheltek és sokszor késve reagálnak a betegségromlásra.

A krónikus betegségek fogalma az orvostudomány egyik legkiterjedtebb kategóriáját jelöli: ide tartoznak mindazok az állapotok, amelyek legalább három hónapig fennállnak, és amelyek rendszeres orvosi felügyeletet, gyógyszeres kezelést, illetve életmódbeli alkalmazkodást igényelnek. A Világegészségügyi Szervezet (WHO) adatai szerint a nem fertőző krónikus betegségek köztük a szív és érrendszeri kórképek, a 2-es típusú cukorbetegség, a krónikus obstruktív tüdőbetegség és a daganatos megbetegedések felelősek a globális halálozás több mint hetven százalékáért. Magyarországon a helyzet különösen aggasztó: a keringési rendszer megbetegedései és az anyagcsere-zavarok a vezető halálokok között szerepelnek, miközben az egészségügyi ellátórendszer kapacitása és a rendelkezésre álló humán erőforrás egyre nagyobb nyomás alatt van. Ez az összefüggés önmagában is indokolja, hogy az informatika és a mérnöki tudományok eszközkészletét aktívan bevonják a krónikus betegségek menedzsmentjébe.

2.1.1 A jelenlegi rendszer hiányosságai

A hagyományos alapú módszerek és ellátási modellek, amelyben az egészségügyi ellátás szükséges több esetben csak akkora kerül sor mikor már a tünetek megjelentek vagy súlyosbodtak egyre kevésbé képes hatékonyan kezeli a növekvő betegszámot, valamint ellátásukat. Ennek következtében szükségessé váltak olyan új módszerek és eszközök alkalmazása, amelyek lehetővé teszik az állapot folyamatos nyomon követését, valamint a minél hamarabb történő orvosi beavatkozást.

A reaktív ellátási szemlélet mögött részben strukturális tényezők állnak. Az alapellátás és a szakellátás közötti koordináció hiányosságai, a háziorvosok túlterheltsége, valamint a szakrendelőkre jellemző hosszú várakozási idők mind azt eredményezik, hogy a krónikus beteg csak akkor kerül orvos elé, amikor állapota már kritikussá vált. Egy 2022-ben az Egészségügyi Világszervezet által közzétett elemzés rámutatott, hogy az időben elvégzett prevenciós beavatkozások, mint a korai figyelmeztetőjelek azonosítása és a terápia időbeni módosítása akár harminc-negyven százalékkal csökkenthetik a súlyos szövődmények kialakulásának kockázatát. Ez az adat egyértelműen alátámasztja, hogy az egészségügyi ellátórendszernek proaktívabb szemléletre van szüksége, amelynek technológiai feltétele a folyamatos, megbízható adatgyűjtés és a valós idejű elemzés.

2.1.2 Az IoT technológia előnyei

Az Internet of Things (IoT) technológia megjelenése, valamint gyors fejlődése miatt új megközelítést biztosít az egészségügyi felügyeletben. Az IoT olyan eszközök hálózatát jelenti,

amelyek képesek adatokat gyűjteni, továbbítani és feldolgozni emberi beavatkozás nélkül. Az egészségügyben ez a technológia olyan eszközökből, szenzorokból, kommunikációs modulokból és adatfeldolgozásból áll, amelyek folyamatosan képesek monitorozni a páciens életfontosságú paramétereit. Ide tartoznak a pulzusszám, vérnyomás, vércukorszint, testhőmérséklet, SpO₂ szint és akár az aktivitási adatok vagy alvásminőség. **(Islam et al., 2020)** *(Az IoT eszközök működésére és az adatok típusaira vonatkozó átfogó tanulmány).*

A problémafelvetés alapja, hogy a hazai rendszerben jelenleg főként manuálisan történnek az adatgyűjtés, valamint személyes orvosi vizsgálatok útján történő páciens állapot követés. Bár egyes kórházakban elérhetőek korszerű diagnosztikai eszközök, azok sokszor nem képesek folytonos monitorozásra, emellett az otthoni felügyelet technológiai támogatottsága is hiányos. Mindezt azt eredményezi, hogy számos krónikus beteg esetében állapotromlás felismerése, illetve a dolgozók terhelése is jelentős, mivel nincs lehetőség automatizált feldolgozásra, valamint arra történő beavatkozásra. A betegek ellátásában nem csak technológiai eredetű hiányosság tapasztalható, hanem strukturális és módszertani okokra is visszavezethető. Az egészségügyi ellátórendszerben a betegutak szervezése gyakran reaktív jellegű, ami azt jelenti, hogy a beavatkozások döntő többsége akkor történik, amikor már túl késő és a beteg állapota már instabil vagy esetleg szövődményeket mutat. Ez nem csak az ellátás költségeit növeli, ugyanis kórházi felvételek vagy esetleg sürgősségi beavatkozás és az intenzív ellátás költségesebb, mint a korai fázisú kezelés. A beteg életminőségét is rontja, hiszen már a kialakult szövődmények sok esetben visszafordíthatatlanok.

Az IoT technológia egészségügyi alkalmazásának egyik legfontosabb elméleti megalapozása Kevin Ashton nevéhez fűződik, aki 1999-ben először használta az „Internet of Things” kifejezést. Az azóta eltelt évtizedekben a technológia fejlődése lehetővé tette, hogy az elmélet valódi klinikai alkalmazásokká érjen. A 2010-es évektől kezdve az okoseszközök miniaturizációja, az akkumulátor-technológia fejlődése és a vezeték nélküli kommunikáció sebességének növekedése együttesen teremtette meg azt az infrastrukturális háttérrel, amely az egészségügyi IoT-rendszerek széles körű elterjedéséhez szükséges volt. Ma már nemcsak a csuklón viselhető pulzuszámoló, hanem beültethető szívritmus-monitorok, tapaszra rögzíthető EKG-elektrodák és intelligens inzulinpumpák is elérhetők a piacon, jelezve, hogy az IoMT nem csupán ígéretes kutatási irány, hanem a mindennapos betegellátás szerves részévé válik.

2.1.3 Az otthoni monitorozás problémái

Az otthoni betegmonitorozás jelenlegi formája Magyarországon nagyrészt a beteg önbevallására, valamint alkalmi orvosi vizsgálaton keresztül működik. A beteg papír alapú naplót vezet a vérnyomásról, vércukrukra, amelyet csak a következő vizsgálat alkalmával tudnak bemutatni az orvosuknak. Ez a módszer több szempontból is hibás, egyrészt az adatok időbeli felbontása, másrészt a beteg hibázhat a mérések elvégzésekor vagy a dokumentálás során, harmadrészt pedig az orvos csak egy-egy pillanatnyi állapotot adathalmaz alapján fog tudni csak dönteni. Az ilyen típusú adatgyűjtés nem alkalmas betegség azonosítására vagy korai riasztásra. (Pal et al., 2021) *(A hagyományos módszerek és az IoT korai riasztásának összevetésére).*

Ezzel szemben az IoT-alapú megoldások folyamatos, objektív és automatikus adatgyűjtést tesz lehetővé, amelyek időfelbontása akár percekre is fel lehet darabolni. A szenzorok által rögzített nyers adatok, mint például a vérnyomásérték, pulzusszám, SpO₂ szintje vagy esetleg vércukorszint napi görbéje. Nem csak az, hogy pontosabb klinikai értéket vagy képeket adnak, hanem előre jelző információkat is tudnak adni az orvosnak. Egy jól kalibrált rendszer már napokkal a tünetek megjelenése előtt jelezheti a szívelégtelenséget, pulzusszám emelkedését, valamint vérnyomás csökkenését.

A hagyományos önbevallás-alapú adatgyűjtés egyik kevésbé tárgyalt, de klinikailag kiemelkedően fontos hiányossága az úgynevezett „fehér köpeny effektus”, amelynek során a beteg vérnyomása az orvosi rendelőben mért állapotban szisztematikusan magasabb, mint a mindennapi életben. Ez a jelenség téves terápiás döntésekhez vezethet: a valóban szükségesnél erősebb vérnyomáscsökkentő kezelés alkalmazásához, vagy éppen ellenkezőleg, a rejtett hipertónia felismerésének elmulasztásához. Az IoT-alapú, otthoni körülmények között végzett folyamatos mérés kiküszöböli ezt a torzítást, mivel a páciens a megszokott környezetében, szabad, természetes körülmények között kerül monitorozásra. A klinikai irodalomban ezt ambuláns vérnyomásmonitorozásnak (ABPM) nevezik, és a vonatkozó irányelvek egyre nagyobb hangsúlyt fektetnek alkalmazására a terápiás döntések megalapozásában.

2.1.4 Társadalmi és földrajzi akadályok

Tapasztalatból beszélve az IoT-rendszerek alkalmazása különösen fontossá válik egy vidéki szolgáltatott területeken, ahol az orvosi ellátáshoz hozzáférés igencsak korlátozott számunkra. Ezeken a területeken a betegek gyakran csakis hosszas utazással, ami nem csak idő igényes, de ha többször is megkell járni akkor akár nagyobb költségek árán tudnak csak megfelelő szakrendeléshez hozzájutni. Egy IoT alapú távmonitorozás rendszer ebben az esetben nem csupán

kényelmi szolgáltatás, hanem valós alternatíva lehet a személyes orvos-beteg találkozásonak. A rendszer által gyűjtött adatokat a háziorvos vagy a gondozószolgálat távolról is ellenőrizheti és amennyiben eltérést észlelnek telefon vagy esetleg videokonferenciával konzultálni tudnak, ha szükséges a beavatkozás. **(e-Health Connect, 2022)(Ray et al., 2021).** *(A távfelügyelet előnyeire vidéki környezetben).*

Ugyanakkor a technológia bevezetése nem megy zökkenőmentesen sem. Az egyik legnagyobb akadályt a betegek digitális tudás hiánya, valamint az idősödő népesség technológiával szembeni bizalmatlanság. A 65 év feletti korosztály közül sokan idegenek az okoseszközök használatától, tartva attól, hogy túl bonyolultak, nem megbízhatóak, valamint akár veszélyeztetik személyes adataikat. Ez a probléma kezelhető lenne egy akár internet lévő teljességében ingyenes, megfelelő oktatással, felhasználóbarát és egyszerű interfészek kialakításával és nem utolsósorban a betegek főként az idősebbek, fokozatos bevonásával a technológia világába. Egy másik, gyakran felmerülő probléma az adatbiztonság és az adatvédelem kérdése. Az egészségügyi adatok kiemelt védelmet élveznek mind a hazai, mind az uniós jogszabályok szerint. Így egy rendszer tervezése során különös figyelmet kell fordítani a titkosításra, hozzáférés kezelésére és adattárolás módjára. A betegeknek tudniuk kell, hogy az adataikhoz, csak az arra jogosult orvosok férhetnek hozzá, amiket természetesen semmi körülmények között nem használhatnak fel kereskedelmi célokra vagy harmadik fél számára semmilyen körülmények között nem továbbítják. A GDPR-ral való teljes körű összhang biztosítása érdekében tartalmaznia kell egy átlátható hozzájárulási folyamatot, amelyben a beteg pontos és precíz tájékoztatást kap arról, milyen adatokat gyűjtenek, hogyan tárolják el azokat és milyen célból használják fel.

Nem utolsósorban, az IoT alapú megoldások bevezetésének gazdasági vonzata is jelentős. Bár a kezdeti beruházási költségek magasak lehetnek hosszú távon azonban a rendszer megtérülhet. A kórházi újrafelvételi arány csökkenése, a sürgősségi ellátás csökkent igénybevétele, gyógyszeres terápiák hatékonyabbá válása és a legfontosabb, a betegek életminőségének növekvő javulása. Ezek mind olyan tényezők, amelyek összességében csökkenthetik az egészségügyi rendszer terheit. Mindezek alapján megállapítható, hogy az IoT alapú monitoring rendszerek bevezetése nem csupán technológiai kérdés, hanem összetett társadalmi, gazdasági és szabályozási kihívás is. A kulcsa a technológia emberközpontú tervezése, betegek aktív bevonása, egészségügyi szakemberek digitális felkészültség fejlesztése, valamint egy olyan keret kialakítása, amely biztosítja az adatbiztonságot, az átláthatóságot és a fenntarthatóságot.

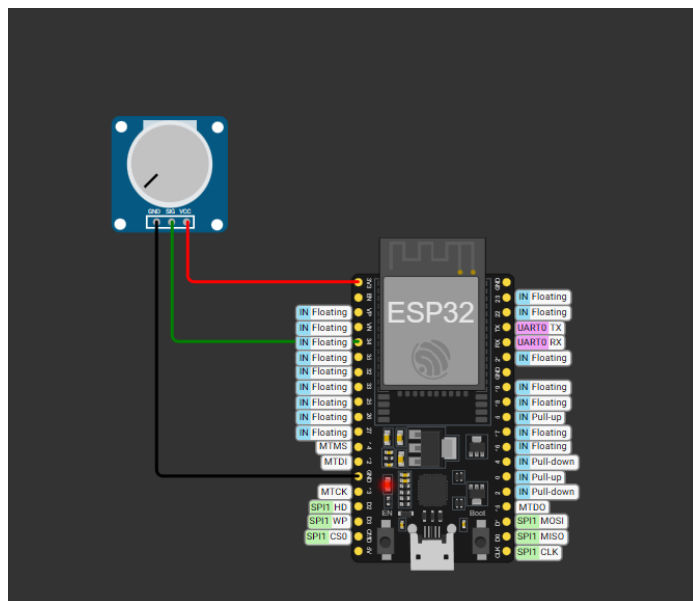
2.2 IoT-technológia az egészségügyben

Az IoT-rendszerek működése három fő rétegre épül: érzékelő réteg, hálózati réteg és alkalmazási réteg. Az érzékelő rétegben különböző szenzorok gyűjtik az adatokat, amelyek vezeték nélküli kommunikációs protokollon (pl. Bluetooth, Wi-Fi) keresztül jutnak el a feldolgozási egységhez. A hálózati réteg végzi az adatátvitelt, míg az alkalmazási réteg felelős az adatok feldolgozásáért, tárolásáért és a felhasználó felé történő megjelenéséért. **(Ray et al., 2021)** *(Az IoT architektúra 3 rétegének szakirodalmi alátámasztására)* Ez a réteg alakítja át a nyers adatokat értelmezhető információvá, például grafikonná vagy riasztási üzenettké, amelyeket az orvosokat és a betegek használnak. Az architektúra hatékony működéséhez elengedhetetlen a rétegek közötti zökkenőmentes kommunikáció és az alacsony késleltetési idő.

Az egészségügyi IoT-architektúra három rétege nemcsak funkcionálisan, hanem biztonsági szempontból is elkülönül egymástól. Az érzékelő rétegben keletkező adatok különösen sebezhetők, hiszen a viselhető eszközök fizikailag a beteg testén vannak, és elveszhetnek vagy megromolódhatnak. A hálózati rétegben az adatátvitel titkosítása kritikus fontosságú, mivel a nem védett kommunikáció lehetőséget adhat illetéktelen hozzáférésre. Az alkalmazási rétegben pedig a hozzáférés-vezérlés és a szerepkör alapú jogosultsági rendszer biztosítja, hogy az orvosi adatokhoz kizárólag az arra feljogosított személyek férhessenek hozzá. E háromszintű biztonsági megközelítés tudatos alkalmazása a jelen szakdolgozatban bemutatott rendszer tervezésének is alapelve, összhangban a GDPR és az egészségügyi adatvédelmi előírásokkal.

2.3 Viselhető szenzorok és kommunikációs protokollok

Az IoT alapú egészségügyi rendszerek (IoMT – Internet of Medical Things) alapkövét a viselhető szenzorok (wearables) és az azokat összekötő kommunikációs technológiák jelentik. A krónikus betegek monitorozása során a legfontosabb szempont a „*non-invasív*” mérés, azaz a bőr megsértése nélküli adatgyűjtés, amely biztosítja a páciens komfortérzetét a mindennapi használat során. **(Al-Garadi et al., 2020)** *(A következő generációs viselhető szenzorokra hivatkozva).*



A fejlesztés során használt IoT mérőeszköz (Saját készítés)

A piacon jelenleg elérhető leggyakoribb szenzortípusok az alábbiak:

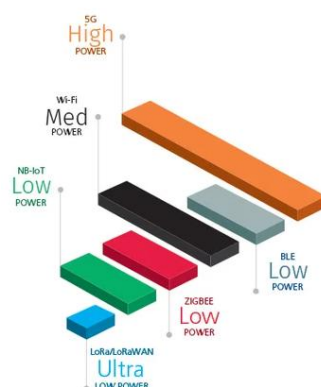
- **Optikai Szenzorok (PPG):** A fotopletizmográfia elvén működő szenzorok (zöld vagy vörös LED fénnel) mérik a véráramlását. Ezeket alkalmazzák a pulzusszám és a véroxigénszint (SpO2) meghatározására. Bár kényelmesek, a mozgásból eredő zajok (motion artifacts) könnyen pontatlanságot okozhatnak.
- **Elektrokémiai szenzorok:** Elsősorban a cukorbetegség számára fejlesztett, szöveti glükózmonitorozó (CGM) rendszerekben használatosak. Ezek a bőr alá helyezett apró szál segítségével mérik a szövetközi folyadék cukorszintjét, kiváltva a napi többszöri ujjbegyszúrást.
- **Inerciális mérőegységek (IMU):** Gyorsulásmérők és giroszkópok kombinációja, amelyek a beteg fizikai aktivitását (lépésszám) és testhelyzetét figyelik. Különösen fontosak az idősellátásban az esések automatikus detektálására. (Pal et al., 2021). *(A szenzorok betegségeszlelésben betöltött szerepére).*

A szenzorok által összegyűjtött adatok továbbításához megfelelő protokollokra van szükség. A választást mindig az energiahatékonyság és a hatótávolság közötti kompromisszum határozza meg:

1. **Rövid hatótávolságú (PAN) technológiák:** A legelterjedtebb a *Bluetooth Low Energy* (BLE), amely lehetővé teszi, hogy a viselhető eszközök hetekig működjenek elemcsere nélkül. A BLE segítségével az eszközök a beteg okostelefonjához (mint átjáróhoz) kapcsolódnak.

Hasonló célra használható a *Zigbee* protokoll is, amely a főként az okosotthon-alapú környezeti szenzoroknál (pl. mérleg, mozgásérzékelő) jellemző.

2. Hosszútávolságú (WAN) technológiák: Amennyiben nincs elérhető okostelefon vagy Wi-Fi hálózat, az *LPWAN* (*Low Power Wide Area Network*) megoldások, mint például az NB-IoT vagy a LoRaWAN kerülnek előtérbe. Ezek a kilométeres távolságba képesek adatot küldeni minimális energiafogyasztással, ideálisak vidéki környezetben élő betegek távfelügyeletére. (Islam et al., 2020). (*A kommunikációs szabványok áttekintésére*).



1. Ábra: Az IoMT kommunikációs technológiáinak összehasonlítása (Hatótáv vs. Adatsebesség)]

2.4 Módszertani kihívások az adatkezelésben

Az egészségügyi adatok gyűjtése, továbbítása és feldolgozása számos módszertani nehézségbe ütközik, amelyek megoldása nélkül nem hozható létre megbízható rendszer.

Az elsődleges kihívás az adatok **minősége és tisztítása**. Ellentétben a klinikai környezettel, ahol a beteg nyugalmi állapotban van, az otthoni monitorozás során a napi tevékenységek zajt visznek a mérésekben. Egy kézmosás vagy séta során a pulzusmérő optikai szenzor elmozdulhat, téves adatokat generálva. Ezért a rendszertervezés során elengedhetetlen a szoftveres szűrés algoritmusok (pl. mozgó átlagolás, Kalman-szűrő) alkalmazása még az adatbázisba kerülés előtt. (Islam et al., 2020)(Ray et al., 2021). (*A zajszűrésre és algoritmusokra*).

A második kritikus pont az **interoperabilitás**, vagyis a rendszerek közötti átjárhatóság hiánya. A különböző gyártók (Omron, Fitbit, Apple) gyakran zárt, egyedi adatszerkezeteket használnak. Ahhoz, hogy egy orvosi rendszer egységes képet kapjon a betegről, szabványosított formátumokra van szükség. Ezen téren a legmodernebb megközelítést a HL7 FHIR (Fast Healthcare

Interoperability Resources) szabvány jelenti, amely API-alapú kommunikációt tesz lehetővé a különböző egészségügyi rendszerek között. *(Az interoperabilitásra és kompatibilitásra vonatkozó cikk).*

Végül meg kell említenem a **„Big Data” és a valós idejű feldolgozás** kérdését. Egyetlen beteg EKG-monitorozása naponta több gigabájt adatot generálhat. Ekkora adatmennyiség folyamatos felhőbe küldése nagy sávszélességet igényel és késleltetést okozhat. A modern módszertant ezért a *Fog Computing* vagy *Edge Computing* irányába mozdul el, ahol az adatok előelemzése már a beteg közelében (pl. a telefonon) megtörténik, és csak a releváns események vagy az aggregált riportok kerülnek továbbításra a központi szerverre. **(Kumar és Patel, 2022).** *(Kifejezetten az Edge Computing valós idejű feldolgozására).*

Az adatkezelési módszertani kihívások kezelése szorosan összefügg az egészségügyi szoftverek minőségbiztosítási követelményeivel. Az orvostechonikai eszközökre vonatkozó európai uniós MDR (Medical Device Regulation, EU 2017/745) rendelet szigorú előírásokat támaszt az adatfeldolgozó algoritmusokkal szemben: a szoftvernek igazolhatóan validált és verifikált folyamatokon kell alapulnia. Ez azt jelenti, hogy az adattisztítási és zajszűrési eljárásokat nemcsak implementálni kell, hanem dokumentálni és tesztelni is, bizonyítva, hogy az algoritmus valóban azt méri, amit mérni szándékoznak. A jelen szakdolgozat prototípus-szintű rendszere ugyan nem törekszik teljes MDR-megfelelőségre, ugyanakkor az adatvalidációs logika tudatos tervezése negatív értékek kiszűrése, biológiailag lehetetlen adatok elutasítása ezt az elvárást tartja szem előtt.

Az interoperabilitás kérdése különösen releváns a magyarországi egészségügyi informatikai kontextusban. Az Elektronikus Egészségügyi Szolgáltatási Tér (EESZT) hazai bevezetésével megnyílt az igény arra, hogy az otthoni monitorozó eszközök által gyűjtött adatok integrálhatók legyenek az országos betegnyilvántartási rendszerbe. Ennek technikai feltétele éppen az a szabványalapú adatcsere, amelyet a HL7 FHIR protokoll tesz lehetővé. A jövőbeni fejlesztések egyik legígéretesebb iránya ezért az EESZT-kompatibilis adatexport megvalósítása, amely lehetővé tenné, hogy az IoT-rendszer által mért adatok közvetlenül bekerüljenek a páciens digitális egészségügyi dokumentációjába, ezzel is erősítve az ellátás kontinuitását.

2.5 Rendszertípusok összehasonlítása

Az egészségügyi monitoring rendszereket felhasználási környezetük alapján három kategóriába sorolhatjuk:

1. Kórházi rendszerek: Zárt hálózaton működnek, magas megbízhatósággal és pontossággal. Hátrányuk a helyhez kötöttség és a magas költség.

2. Otthoni (Consumer) rendszerek: A kereskedelmi forgalomban kapható okoseszközökre épülnek. Előnyük a kényelem és az alacsony ár, hátrányuk a mérések alacsonyabb klinikai pontossága és az orvosi felügyelet hiánya.

3. Hibrid rendszerek: A jelen szakdolgozat célja is egy ilyen rendszer koncepciója. A hibrid modell ötvözi az otthoni mérések kényelmét a klinikai adatfeldolgozás szigorával. A rendszer képes alkalmazkodni: kritikus állapot észlelésekor a mintavételezés sűrűsége automatikusan növelhető, megközelítve a kórházi megfigyelés szintjét, míg stabil állapotban energiatakarékos módra vált.

A három rendszertípus összehasonlítása rávilágít arra, hogy az optimális megoldás nem feltétlenül a legtöbb funkciót kínáló, hanem a konkrét klinikai kontextushoz legjobban illeszkedő rendszer. Egy stabil, ambuláns ellátásban kezelt hipertóniás beteg számára elegendő lehet egy egyszerű, olcsó okosórás vérnyomásmérő, míg egy instabil szívelégtelenségben szenvedő páciens esetén a folyamatos, kórházi szintű monitorozás elengedhetetlen. A rendszertípusok közötti választás tehát klinikai döntés, amelyet az orvosnak kell meghoznia a beteg állapota és kockázati profilja alapján. Az informatikai rendszer feladata, hogy ezt a döntést rugalmasan támogassa: lehetővé tegye a különböző monitorozási intenzitások beállítását, és biztosítsa az adatok folyamatos, megbízható elérhetőségét az egészségügyi szakember számára.

A 2. fejezet áttekintése összességében megmutatja, hogy az IoT-alapú egészségügyi monitoring nem egyetlen technológia, hanem egy komplex, egymásra épülő rendszerelemekből álló ökoszisztéma. A krónikus betegellátás kihívásainak azonosításától a szenzortechnológián és kommunikációs protokollokon át az adatkezelési módszertanig és a rendszertípusok összehasonlításáig terjedő áttekintés megteremti azt az elméleti keretet, amelyre a szakdolgozat gyakorlati fejlesztési munkája támaszkodik. A következő fejezet a piaci és jogi környezet feltárásával egészíti ki ezt az alapozást, biztosítva, hogy a tervezett rendszer ne csupán technikailag megalapozott, hanem jogilag is konform és a valós piaci viszonyokhoz illeszkedő megoldást képviseljen.

3. Piaci Körkép, biztonsági kihívások és jogi szabályozás

Egy IoT alapú egészségügyi rendszer fejlesztése nem történhet légüres térben. A technológia tervezése mellett elengedhetetlen a piaci környezet ismerete, valamint azon szigorú biztonsági és jogi keretek betartása, amelyek az Európai Unióban a betegadatok kezelését szabályozzák

A távmonitorozási jelenleg két, egymástól elkülönülő szegmensre osztható: a fogyasztói elektronikai eszközökre és a minősített orvostechnikai eszközökre.

A globális digitális egészségügyi piac dinamikus növekedést mutat: a Grand View Research 2023-as piacelemzése szerint az IoT-alapú egészségügyi megoldások piaca 2030-ra várhatóan meghaladja a 534 milliárd dolláros értéket, évi átlagos növekedési ütemmel (CAGR) közel 19 százalékon. Ezt a bővülést elsősorban a krónikus betegségek terjedése, az idősödő népesség arányának növekedése és a telemedicina iránt megnövekedett igény hajtja. Az európai piacon belül Magyarország a digitális egészségügy terén felzárkózó pozícióban van: bár az EESZT kiépítése alapvető infrastrukturális lépést jelentett, a távmonitorozási megoldások integrált, rendszerszintű alkalmazása még nem terjedt el széles körben az egészségügyi alapellátásban. Ez egyszerre jelent kihívást és lehetőséget: a fejlesztési tér nagy, ugyanakkor a piac befogadókészsége és a szabályozói környezet is formálódóban van.

A **fogyasztói piacot** olyan technológia óriások dominálják, mint az *Apple (Apple watch)*, a *Google (Fitbit)*, a *Garmin* vagy *Samsung*.

- **Előnyök:** Ezek az eszközök rendkívül felhasználóbarátok, esztétikusak, és gamifikáció (játékosítás) segítségével motiválják a felhasználót az egészséges életmódra. Képesek pulzus, véroxigén, aktivitás és alvás mérésére.
- **Hátrányok:** A legnagyobb probléma az adatok „silószerű” tárolása. Az Apple Health vagy Google Fit adatai nehezen integrálhatók a magyar kórház rendszerekbe (EESZT, HIS). Az orvosok nem férnek hozzá ezekhez valós időben, a betegek gyakran csak képernyőmentéseket vagy Excel táblázatokat tudnak mutatni rendelésen. Emellett a mérések pontossága sokszor nem éri el a diagnosztikai szintet.

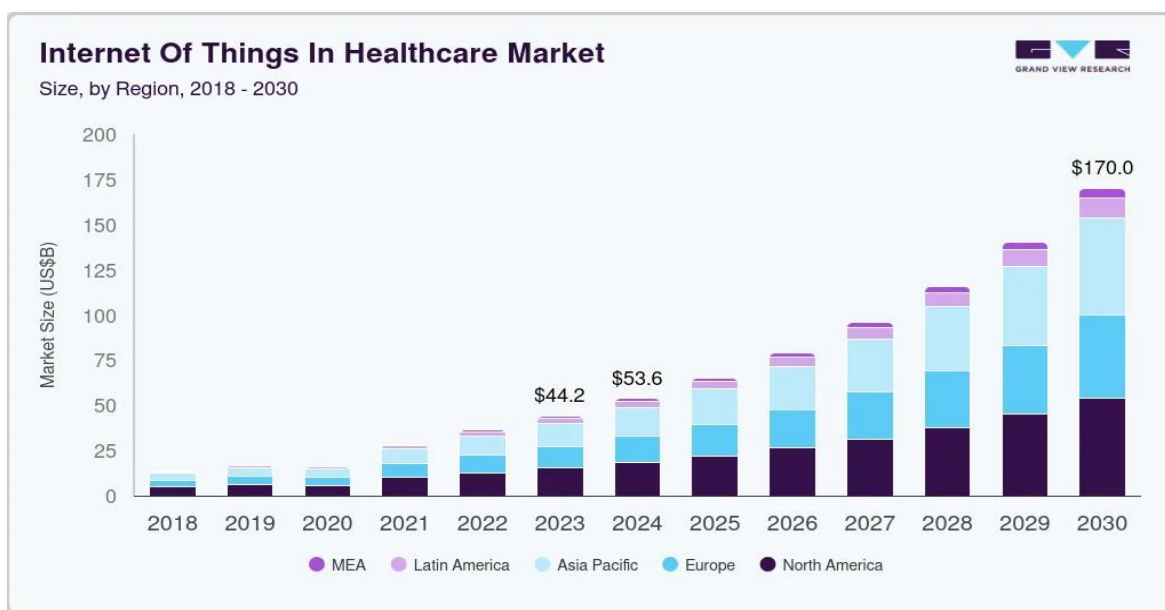
Az **orvostechnikai szoftverek és eszközök** (pl. *Omron Connect*, *Abbott FreeStyle Libre*, *Withings Health Mate*) kifejezetten betegségek menedzselésére készültek.

- **Előnyök:** Klinikailag valós mérési pontosság (CE és FDA minősítés), megbízhatóság.

- **Hátrányok:** Ezek a megoldások gyakran drágák és nem alkotnak egységes ökoszisztémát. Egy betegnek, aki cukorbetegséggel és magas vérnyomással is küzd, jelenleg két-három különböző applikációt kell használnia párhuzamosan, ami rontja a terápiahűséget (adherence).

A szakdolgozatban fejlesztendő rendszer célja ezen hiányosságok áthidalása: egy olyan platformfüggetlen webes felület létrehozása, amely képes aggregálni az adatokat, és egyetlen, áttekinthető műszerfalon (dashboard) megjeleníteni azokat az orvosok számára, ötvözve az olcsóbb szenzorok elérhetőségét az orvosi felügyelet biztonságával.

A piaci szegmensek elemzése rávilágít arra az alapvető feszültségre, amely a fogyasztói és az orvostechnikai megközelítés között fennáll. A fogyasztói eszközök erőssége a tömegpiaci elérhetőség és a felhasználói élmény, gyengeségük viszont az orvosi integráció hiánya. Az orvostechnikai eszközök klinikai megbízhatósága vitathatatlan, ugyanakkor fragmentált ökoszisztémájuk és magas árak komoly akadályt jelent a széles körű elterjedés előtt. A jelen szakdolgozatban fejlesztett ChronoCare rendszer tudatosan igyekszik e két szélsőpont között egyensúlyt teremteni: egy olyan nyílt, webalapú architektúrát valósít meg, amely képes szimulálni és a jövőben valódi szenzoradatokat fogadni, azokat egységes felületen megjeleníteni, és az orvos számára valós idejű klinikai áttekintést biztosítani. Ez a megközelítés összhangban van azzal az iparági trenddel, amelyet a szakirodalom „platform-agnosztikus egészségügyi adat-aggregációnak” nevez.



2. ábra A globális IoT egészségügyi piac növekedése

Az IoT-alapú egészségügyi rendszerek biztonsági kihívásai nem csupán jogi, hanem műszaki és szervezeti dimenzióban is megjelennek. Az egészségügyi kiberbiztonság területén az elmúlt években drámai emelkedés figyelhető meg a célzott támadások számában: az IBM Security 2023-as jelentése szerint az egészségügyi szektor átlagos adatszivárgási kára elérte a 10,9 millió dollárt, ami a legmagasabb értéket jelenti bármely iparág körében. Az IoT-eszközök különösen sebezhetők, mivel sokszor korlátozott feldolgozóteljesítményük miatt nem képesek összetett titkosítási eljárások futtatására, frissítéseik rendszertelenek, és fizikailag is hozzáférhetők. Egy egészségügyi IoT-rendszer biztonsági tervezésekor ezért elengedhetetlen a „defence in depth” elv alkalmazása, amely szerint egyetlen biztonsági réteg sem elegendő a védelem több, egymástól független szinten kell megvalósuljon, a szenzortól az adatbázisig.

3.1 Jogi háttér (GDPR és egészségügyi adatvédelem)

Európai uniós tagállamként Magyarországon minden személyes adatot kezelő rendszernek meg kell felelnie az Európai Parlament és a Tanács (EU) 2016/679 rendeletének, azaz a **GDPR**-nak (Hassan, 2022). *(Magára a hivatalos jogszabályra).*

Az egészségügyi adatok a GDPR 9. cikke értelmében „**különleges kategóriájú személyes adatnak**” minősülnek. Kezelésük főszabály szerint tilos, kivéve, ha szigorú feltételek teljesülnek (Spring Framework, 2024). *(Az EU adatvédelmi szabályozására).*

A tervezett rendszer esetében a jogszerű adatkezelés pillérei a következők:

1. **Kifejezett hozzájárulás (Explicit Consent):** A betegnek egyértelmű, tájékoztatáson alapuló, aktív cselekedettel (nem előre bepipált négyzet!) kell hozzájárulnia adatai monitorozásához. A tájékoztatónak tartalmaznia kell az adatkezelés célját, időtartamát és az adatokhoz hozzáférők körét.
2. **Adattakarékosság (Data Minimization):** Csak a cél eléréséhez feltétlenül szükséges adatokat szabad gyűjteni. Például egy vérnyomás-monitorozó alkalmazásnak nem szükséges tudnia a beteg pontos GPS koordinátáit vagy hozzáférnie a telefon névjegyzékéhez.
3. **Beépített adatvédelem (Privacy by Design):** A szoftver architektúráját úgy kell kialakítani, hogy az alapértelmezetten védje a privát szférát. Ennek egyik legfontosabb eszköze a **pszeudonimizálás**: az orvosi mérési adatok és a beteg személyazonosító adatainak (név, TAJ szám) logikai szétválasztása az adatbázisban. Így egy esetleges adatszivárgás esetén a támadó csak anonim számsorokat lát, amelyeket nem tud konkrét

személyhez kötni (Pal et al., 2021)." (*A biztonsági és adatvédelmi mechanizmusokra, pszeudonimizációra*).

4. **Az érintettek jogai:** A rendszernek technikailag biztosítani kell a beteg számára a hozzáférést saját adataihoz, az adatok hordozhatóságát (exportálását), valamint az „elfeledtetéshez való jogot”, azaz az adatok törlését a szolgáltatás megszűnésekor.

A GDPR által meghatározott adatvédelmi elvek és az érintetti jogok nem csupán jogi kötelezettségeket írnak elő, hanem egyúttal a bizalmi infrastruktúra építőköveit is jelentik. Egy krónikus beteg ugyanis csak akkor veszi igénybe következetesen és hosszú távon a monitorozó rendszert, ha bízik abban, hogy adatai biztonságban vannak és azokkal nem élnek vissza. A terápiahűség (adherencia) és az adatvédelem tehát szorosan összefüggenek: egy átlátható, betegközpontú adatkezelési modell nemcsak jogi megfelelést biztosít, hanem közvetlenül hozzájárul a rendszer klinikai hatékonyságához is. A jelen szakdolgozatban fejlesztett ChronoCare rendszer ezért a GDPR elveit nem akadályként, hanem a tervezés kiindulópontjaként kezeli a pszeudonimizálást, a szerepkör alapú hozzáférés-vezérlést és a minimális adatgyűjtés elveit a rendszer architektúrájába épülnek be, nem utólag kerülnek hozzáadásra.

A 3. fejezet összességében megmutatja, hogy egy IoT-alapú egészségügyi rendszer fejlesztése nem szűkíthető le tisztán mérnöki feladatra. A piaci környezet megértése, a biztonsági fenyegetések tudatosítása és a jogi megfelelés biztosítása olyan alapfeltételek, amelyek nélkül a legkifinomultabb technológiai megoldás sem lehet fenntartható vagy etikus. A következő fejezetek a rendszer konkrét tervezési és megvalósítási lépéseit tárják fel, szem előtt tartva az itt azonosított piaci, biztonsági és jogi szempontokat.

4. Rendszer tervezése

Az elméleti alapok és a piaci környezet áttekintése után a szakdolgozat ezen fejezete a fejlesztendő IoT alapú egészségügyi monitoring rendszer konkrét tervezési lépéseit tárgyalja. A szoftverfejlesztés életciklusának (SDLC) tervezési fázisa kritikus fontosságú, mivel itt kerülnek definiálásra azok a specifikációk, amelyek biztosítják, hogy a végeredmény megfeleljen mind a felhasználói igényeknek, mind a szigorú adatvédelmi előírásoknak.

A tervezés során a vízesés modell és az agilis módszertan elemeit ötvöztem: a követelmények kezdeti rögzítése (specifikáció) statikus, míg a funkciók finomhangolása iteratív módon történt.

A vízesés modell és az agilis módszertan kombinációjának alkalmazása a szoftverfejlesztési életciklus egyik bevett hibrid megközelítése, amelyet a szakirodalom gyakran „iteratív vízesésként” vagy „félagilis tervezésként” említ. A vízesés modell elemei biztosítják, hogy a rendszer alapvető funkcionális és nem funkcionális követelményei az implementáció megkezdése előtt rögzítésre kerüljenek, megelőzve a tervezési inkonzisztenciákat és az utólagos, költséges módosításokat. Az agilis elvek beépítése ugyanakkor lehetővé teszi a felhasználói visszajelzések és a fejlesztés során felmerülő új szempontok rugalmas kezelését. Ez a kettős megközelítés különösen indokolt egészségügyi szoftverek esetén, ahol a biztonsági és adatvédelmi követelmények előzetes rögzítése elengedhetetlen, miközben a felhasználói élmény és az interfész finomhangolása iteratív folyamat.

4.1 Funkcionális követelmények

A funkcionális követelmények írják le a rendszer viselkedését, azaz azt, hogy a szoftvernek milyen konkrét feladatot kell ellátnia a felhasználói interakciók vagy egyéb események hatására. A rendszerben három fő szerepkört azonosítottam: a **Páciens**, az **Orvost** és magát a **Rendszeradminisztrátort**.

4.1.1 Felhasználói szerepkörök és használati esetek (Use Cases)

1. Páciens

A páciens a rendszer elsődleges adatforrása és kedvezményezettje. Számára a felületnek egyszerűnek és átláthatónak kell lennie.

- **Regisztráció és bejelentkezés:** A páciens képes legyen saját fiókot létrehozni személyes adatainak (név, életkora, TAJ szám) megadásával. A belépés biztonságos hitelesítéssel történjen.
- **Mérési adatok beküldése:** A rendszernek lehetőséget kell biztosítania az adatokat automatikus fogadására (szimulált IoT eszközről) és manuális rögzítésére is (pl: ha a beteg régebbi típusú vérnyomásmérőt használ.)
- **Saját adatok megtekintése:** A páciens valamilyen formában is láthassa saját mérési előzményeit (vérnyomás, vércukor, pulzus).

2. Orvos (Doctor):

Az orvos a felügyelő fél, aki a beérkező adatok alapján döntéseket hoz.

- **Pácienslista kezelése:** Az orvos láthassa a hozzá rendelt betegek listáját.
- **Részletes betegadatok megtekintése:** Egy adott páciens vizuálisan jelezze az orvos számára, ha valamelyik páciensnél a mért érték állépte a kritikus küszöböt

3. Rendszer / Szenzor

- **Adatvalidáció:** A beérkező adatok ellenőrzése (pl. ne lehessen negatív pulzust rögzíteni).
- **Küszöbérték-figyelés:** Minden beérkező adat összevetése az előre definiált normál tartományokkal.
- **Adattárolás:** A validált adatok perzisztens mentése az adatbázisba időbélyeggel ellátva.

4.1.2 Adatfolyam és folyamatleírás

A rendszer működésének központi eleme az adat útja a méréstől a megjelenítésig. A tervezett adatfolyam a következő lépésekből áll:

1. **Adatgenerálás:** A szimulált IoT szenzor JSON formátumban előállítja a mérési adatot (típus, mértékegység, érték, dátum)
2. **API hívás:** MQTT publish üzenetet küld a patient/{patientId}/measurements topicra
3. **Feldolgozás:** A szerver (Controller réteg) fogadja a kérést, a Service réteg elvégzi az üzleti logikai ellenőrzést (jogosultság, határérték ellenőrzés).
4. **Perzisztencia:** Az adatbázis réteg (Repository) elmenti az adatot a relációs adatbázisba.
5. **Visszacsatolás:** Ha az érték kritikus, a rendszer azonnal „WARNING” flag-et (jelzőt) kapcsol a méréshez, amely a frontend oldalon piros kiemeléssel jelenik meg.

Az adatfolyam fenti leírása egyúttal a rendszer hibatűrési modelljét is körvonalazza. Minden egyes lépésnél meghatározható az a pont, ahol a folyamat megszakadhat, és ahol az erre vonatkozó kivételkezelést implementálni szükséges. Amennyiben például a szenzor egy érvénytelen JSON-struktúrát küld, a Controller rétegnek ezt már a beérkezéskor el kell utasítania, és megfelelő HTTP hibaüzenettel kell válaszolnia. Ha az adatbázis-mentés sikertelen, a tranzakciókezelőnek vissza kell görgetnie a műveletet, megakadályozva a részleges adatmentést. Ez a rétegenkénti hibakezelési szemlélet összhangban van a „fail-fast” elvvel, amely szerint a rendszert minél hamarabb és minél egyértelműbb hibaüzenettel kell megállítani, ha valami nem a tervek

szerint alakul. Az egészségügyi szoftverek esetében ez különösen fontos, mivel egy részlegesen mentett, hibás mérési adat félrevezető klinikai döntéshez vezethet.

4.2 Nem funkcionális követelmények

Míg a funkcionális követelmények a „mit” kérdésre válaszolnak, a nem funkcionális követelmények a „hogyan” kéréskört fedik le, meghatározva a rendszer minőségi tulajdonságait.

1. Teljesítmény és válaszidő (Performance):

Egészségügyi rendszerről lévén szó, a késleltetés minimalizálása kulcsfontosságú. A tervezési cél, hogy a felhasználói felület (Dashboard) betöltési ideje ne haladja meg a 2 másodpercet, az API válaszüze pedig 200 ms alatt maradjon átlagos terhelés mellett.

2. Skálázhatóság (Scalability):

Bár a szakdolgozat keretében egy prototípus készül, a tervezésnél figyelembe kell venni a későbbi bővíthetőséget. A rendszer architektúrájának (rétegelt szerkezet) támogatnia kell, hogy a felhasználók számának növekedésével a szerver erőforrásai (CPU, RAM) vertikálisan bővíthetők legyenek, vagy a rendszer később mikró szolgáltatásokra (Microservices) bontható legyen. A tervezett adatbázis-struktúra képesnek kell lennie több ezer páciens több milliónyi mérési adatának kezelésére is.

3. Rendelkezésre állás (Availability)

A rendszernek a nap 24 órájában elérhetőnek kell lennie (99,9%-os uptime), hiszen a krónikus betegek állapota bármikor változhat. Ezt a tervezés során a robusztus hibakezelés (Exception Handling) és a szerveroldali állapotmentesség (Stateless architecture) biztosítja.

4. Biztonság (Security):

A 3. fejezetben tárgyalt GDPR és adatvédelmi elveknek megfelelően:

- Az API végpontok védettek, csak a hitelesített felhasználók érhetik el őket.
- Szerepkör alapú hozzáférés-vezérlés (RBAC – Role Based Access Control): A páciens nem láthatja más páciens adatait, az orvos viszont láthatja a hozzá rendelt betegeket.

A nem funkcionális követelmények teljesíthetőségét az egyes architekturális döntések alapvetően meghatározzák. A 200 ms-os API válaszidő például csak akkor garantálható, ha az adatbázis-lekérdezések megfelelően optimalizáltak: az indexek helyes alkalmazása a PATIENT_ID és

TIMESTAMP mezőkön döntő szerepet játszik a nagy adatmennyiség esetén is gyors keresésben. A 99,9%-os rendelkezésre állás megközelítése a fejlesztési fázisban a szerveroldali állapotmentességgel (stateless architecture) érhető el, amelyet a Spring Boot alapértelmezetten támogat: minden HTTP-kérés önálló, kontextusfüggetlen egységként kezelendő, így a szerver újraindítása sem okoz adatvesztést. A skálázhatósági célok megvalósíthatóságát pedig éppen a rétegelt, moduláris architektúra garantálja, amely lehetővé teszi az egyes komponensek például az adatbázisréteg független cseréjét vagy bővítését a rendszer többi részének módosítása nélkül.

4.3 Adatbázis-tervezés

A rendszer adatainak perzisztens tárolására egy beágyazott, Java-alapú relációs adatbázis-kezelőt, a **H2 database Engine**-t választottam. A fejlesztési fázisban ez a megoldás rendkívül előnyös, mivel nem igényel külön szerverinstallációt, fájl alapú tárolást tesz lehetővé, és könnyen integrálható a Spring Boot keretrendszerrel.

A tervezés során a **szerepkörök szerinti szétválasztás** (Table-per-Concrete-Class stratégia jellegű) mellett döntöttem. Ezt azt jelenti, hogy az orvosok és a páciensek adatai fizikailag külön táblákba helyezkednek el, mivel a hozzájuk tartozó attribútumok jelentősen eltérnek (pl. csak a betegnek van diagnózisa)

Az adatbázis sémája három fő entitásra épül: **DOCTOR** (Orvos) **PATIENT** (Páciens) és **HEALTH_MEASUREMENT** (Egészségügyi mérés).

4.3.1 Adattáblák definíciója és szerkezete

Az alábbiakban részletezem a megvalósított fizikai adatmodell tábláit és azok mezőit.

1. **DOCTOR** tábla:

Ez a tábla tárolja az egészségügyi szakemberek belépési és azonosítási adatait. Az orvosok a rendszerben a felügyelői szerepkört töltik be.

- **ID (elsődleges kulcs):** Egyedi azonosító, amely alapján a rendszer hivatkozik az orvosra.
- **NAME:** Az orvos teljes neve, amely megjelenik a felületen.
- **USERNAME:** A bejelentkezéshez használt egyedi felhasználónév.
- **PASSWORD:** A biztonsági előírásoknak megfelelően titkosítva tárolt jelszó.

2. PATIENT tábla:

A rendszer központi szereplői, a krónikus betegek adatai itt kerülnek rögzítésre. A tábla szerkezete tükrözi a betegellátáshoz szükséges specifikus információkat.

- **ID (elsődleges kulcs):** A páciens egyedi rendszer azonosítója.
- **NAME:** A beteg teljes neve
- **AGE:** Életkor, amely fontos orvosi kockázati tényező lehet.
- **DIAGNOSIS:** A krónikus betegség megnevezése (pl: „II. típusú diabetes, Hipertónia”). Ez a mező segíti az orvost a beérkező adatok kontextusba helyezésében.
- **USERNAME és PASSWORD:** A páciens saját fiókjának belépési adatai, hogy megtekinthesse saját mérési előzményeit.
- **DOCTOR_ID (Idegen kulcs):** Ez a mező teremti meg a kapcsolatot az orvos és a beteg között. A modellben egy **1:N (egy a többhöz)** kapcsolatot valósítottam meg, ami azt jelenti, hogy egy pácienshez egy dedikált kezelőorvos tartozik, aki felelős a monitorozásáért. Ez a megoldás egyszerűsíti a jogosultságkezelést és a felelősségi köröket.

3. HEALTH_MEASUREMENT tábla:

Ez a tranzakciós tábla tárolja az IoT eszköztől vagy manuális bevitelből származó idősoros adatokat. Mivel a rendszer célja a folyamatos monitorozás, várhatóan ez a tábla tartalmazza majd a legtöbb rekordot.

- **ID (elsődleges kulcsa):** A mérés egyedi azonosítója
- **PATIENT_ID (Idegen kulcs):** Hivatkozás a **PATIENT** táblára, amely egyértelműen hozzárendeli a mérést egy konkrét beteghez.
- **TYPE:** A mérés típusa
- **MEASURED_VALUE:** A tényleges mért numerikus érték.
- **TIMESTAMP:** A mérés pontos időpontja. Ez kritikus fontosságú az adatlap megjelenítésén.
- **IS_CRITICAL:** Logikai (Boolean) mező, amelyet a rendszer automatikusan állít be az adatok beérkezésekor. Ha a mért érték átlép egy előre definiált küszöböt, ez a mező **TRUE** értéket vesz fel. Ez teszi lehetővé, hogy az orvosi felületen azonnal, erőforrás-igényes újra számolás nélkül lehessen szűrni a riasztást igénylő esetekre.

4.3.2 Kapcsolatok és integritás

A táblák közötti kapcsolatok biztosítják az adatok konzisztenciáját (Referential Integrity):

1. Orvos – Páciens kapcsolat: A **PATIENT** tábla **DOCTOR_ID** külső kulcsa mutat a **DOCTOR** tábla **ID** mezőjére. Ez biztosítja, hogy minden beteghez létező orvos legyen rendelve. Ha egy orvost törölnénk a rendszerből, az adatbázis megszorításai megakadályozzák az árva rekordok létrejöttét (vagy kaszkádolt törlést, vagy a törlés tiltását eredményezve).

2. Páciens – Mérés kapcsolat: A **HEALTH_MEASUREMENT** tábla **PATIENT_ID** külső kulcsa kapcsolja össze a mérést a beteggel. Ez egy klasszikus szülő-gyermek kapcsolat, ahol egy beteghez tetszőleges számú mérés tartozhat.

Ez az adatbázis-struktúra, bár egyszerűsített (például a "many-to-many" orvos-beteg kapcsolat helyett "one-to-many" modellt alkalmaz), tökéletesen megfelel egy szakdolgozati prototípus követelményeinek. Lehetővé teszi a gyors adat lekérdezést, a kritikus értékek (**IS_CRITICAL**) azonnali szűrését, valamint a személyes adatok és a mérési adatok logikai szétválasztását.

A választott adatmodell a normalizáció első három Boyce–Codd-szintjének (BCNF) elveit követi: minden tábla egyetlen entitást reprezentál, az attribútumok atomiak, és a funkcionális függőségek egyértelműen az elsődleges kulcsokon alapulnak. Ez a normalizált struktúra minimalizálja az adatredundanciát és megakadályozza az anomáliák kialakulását például egy beteg névének módosításakor elegendő egyetlen rekordot frissíteni a **PATIENT** táblában, az összes hozzá tartozó mérési adat automatikusan a naprakész névhez kapcsolódik. Az **IS_CRITICAL** boolean mező denormalizált elemnek tekinthető, mivel értéke a mérési adatból elvben újra számítható azonban ez tudatos tervezési döntés, amely az olvasási teljesítményt helyezi előtérbe az írási normalizáltsággal szemben, összhangban az OLTP (Online Transaction Processing) rendszerek bevált optimalizációs elveivel.

A 4. fejezet összességében lefektette azt a tervezési alapot, amelyre a ChronoCare rendszer teljes megvalósítása épül. A funkcionális és nem funkcionális követelmények pontos definiálása, az adatfolyam részletes leírása és az adatbázisséma gondos kialakítása együttesen biztosítják, hogy a fejlesztési munka ne ad hoc döntéseken, hanem tudatos, dokumentált architektúrális elveken alapuljon. A következő fejezet az elkészült rendszer konkrét implementációját mutatja be, visszautalva az itt meghatározott tervezési döntésekre.

5. Fejezet A rendszer architekturális felépítése és implementációja

A szakdolgozat gyakorlati részének célja egy olyan IoT (Internet of Things) alapú egészségügyi monitoring rendszer, a ChronoCare prototípusának kifejlesztése volt, amely képes krónikus betegek élettani adatainak távoli gyűjtésére, tárolására és valós idejű elemzésére. A rendszer tervezése során az elsődleges szempont a felhasználóbarát felület, az adatbiztonság és az orvosoknak szakmailag releváns riasztási logika implementálása volt. A jelen fejezet részletesen bemutatja az elkészült szoftver architektúráját, a felhasználói szerepkörök kezelését, a vizualizációs megoldásokat, valamint a háttérben futó üzleti logikát, amely a kritikus egészségügyi állapotok detektálásáért felel.

A ChronoCare rendszer a modern webfejlesztésben elterjedt, **Model-View-Controller (MVC)** tervezési mintára épül. Ez az architekturális döntés biztosítja a felelősségi körök szétválasztását: az adatok reprezentációja (Model), a felhasználói felület (View) és a vezérlési logika (Controller) külön rétegekben helyezkedik el, ami növeli a kód karbantarthatóságát és skálázhatóságát (**Spring Framework, 2024**). *(Az MVC architektúra orvosi környezetben való alkalmazására).*

A fejlesztői környezetben (IntelliJ IDEA) a projekt struktúrája az alábbi logikai egységekre tagolódik, követve a Spring Boot keretrendszer konvencióit:

1. Adatmodell réteg (**com.chronocare.chronocare.model**):

Ez a réteg képezi az alkalmazás gerincét. Itt kerültek definiálásra azok a Java osztályok (Entitások), amelyek az adatbázis tábláit reprezentálják. A rendszer három fő entitással dolgozik:

- **Patient (Páciens):** Tárolja a személyes adatokat (név, életkor, TAJ szám), a diagnózist, valamint a belépéshez szükséges hitelesítő adatokat. Egy pácienshez egy orvos és több mérési adat tartozhat.
- **Doctor (Orvos):** Az egészségügyi személyzet adatait tárolja. Az adatbázis-kapcsolatok révén egy orvoshoz listányi páciens rendelhető, biztosítva a „One-to-Many” (egy-a-többhöz) kapcsolatot.
- **HealthMeasurement (Egészségügyi mérés):** Ez felel a szenzoroktól érkező idősoros adatok tárolásáért. Tartalmazza a mérés típusát (pl. vérnyomás, vér-cukor), értékét, időbélyegét és a rendszer által megállapított státuszt (normál vagy kritikus).

2. Adatelérési réteg (...repository):

A közvetlen SQL lekérdezések helyett a rendszer a Spring Data JPA technológiát használja. A repository interfészek biztosítják a kapcsolatot a Java objektumok és a fizikai adatbázis (esetünkben H2 file-based database) között. Ez lehetővé teszi az adatok perzisztens tárolását, így a rendszer újraindítása után sem vesznek el a rögzített páciensadatok.

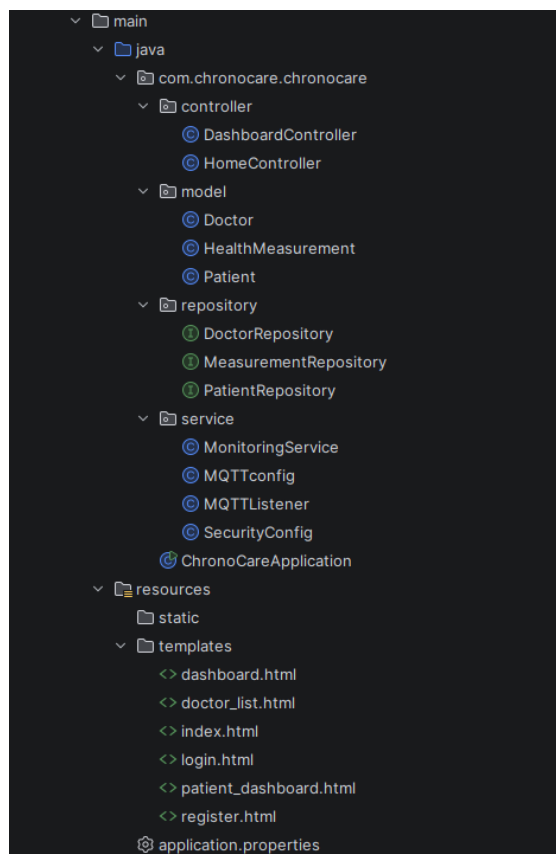
3. Üzleti logika réteg (...service):

A MonitoringService osztály tartalmazza az alkalmazás intelligenciáját. Ez a réteg felelős azért, hogy a beérkező nyers adatokat feldolgozza, validálja (pl. negatív értékek kiszűrése), és összevesse az orvosi határértékekkel. A Controller réteg sosem kommunikál közvetlenül az adatbázissal, minden művelet ezen a szervizrétegen halad át, garantálva az adatbiztonságot.

4. Vezérlő réteg (...controller):

A rendszer „forgalomirányítója”. Két fő vezérlőt különböztetünk meg:

- HomeController: A webes navigációt, a regisztrációt és a bejelentkezési folyamatokat kezeli.
- DashboardController: A páciensek adatlapjának megjelenítéséért és a frontend felé történő adattovábbításért felel.



3. ábra: A ChronoCare projektstruktúrája az IntelliJ környezetben (Forrás: saját szerkesztés, 2025)

5.1. Felhasználói szerepkörök és jogosultságkezelés

Mivel a rendszer érzékeny egészségügyi adatokat kezel (GDPR relevancia), a fejlesztés során kiemelt figyelmet kapott a szerepkörök elkülönítése. A rendszer indításakor egy letisztult nyitóoldal („Landing Page”) fogadja a látogatót, ahol két fő bejelentkezési útvonal közül választhat:

- **Páciens portál:** A betegek számára készült felület. Itt lehetőség van új regisztrációra is, ahol a páciensnek kötelezően meg kell adnia a nevét, életkorát, TAJ számát, valamint ki kell választania kezelőorvosát a rendszerben regisztrált doktorok listájából. Bejelentkezés után a páciens kizárólag a saját méréseit láthatja, más betegek adataihoz nem férhet hozzá.
- **Orvosi adminisztrációs felület:** Az orvosok számára dedikált belépési pont. A sikeres hitelesítést követően az orvos egy összesítő listát (doctor_list.html) lát, amelyben csak a hozzá rendelt páciensek szerepelnek. Ez a szűrés a szerver oldalon történik (Session alapú kezelés), biztosítva, hogy egy orvos véletlenül se lássa más kollégák betegeit. A

listában megjelenik a páciens neve, kora és diagnózisa, valamint egy akciógomb, amelyel megnyitható a részletes adatlap.

5.2. Vizualizáció: A Páciens Műszerfal (Dashboard)

A rendszer központi eleme a Páciens Adatlap, amely ergonómiai szempontok alapján került kialakításra. A felület a Bootstrap CSS keretrendszert használja, így reszponzív, azaz táblagépen és asztali számítógépen is megfelelően jelenik meg. A dashboard három fő szekcióra osztható:

1. Információs fejléc:

Az oldal tetején kék háttérrel kiemelve láthatóak a páciens legfontosabb statikus adatai: Név, Életkor, TAJ szám és az elsődleges Diagnózis. Ez biztosítja, hogy az orvos mindig tisztában legyen azzal, kinek az adatait elemzi éppen, csökkentve a tévesztés kockázatát.

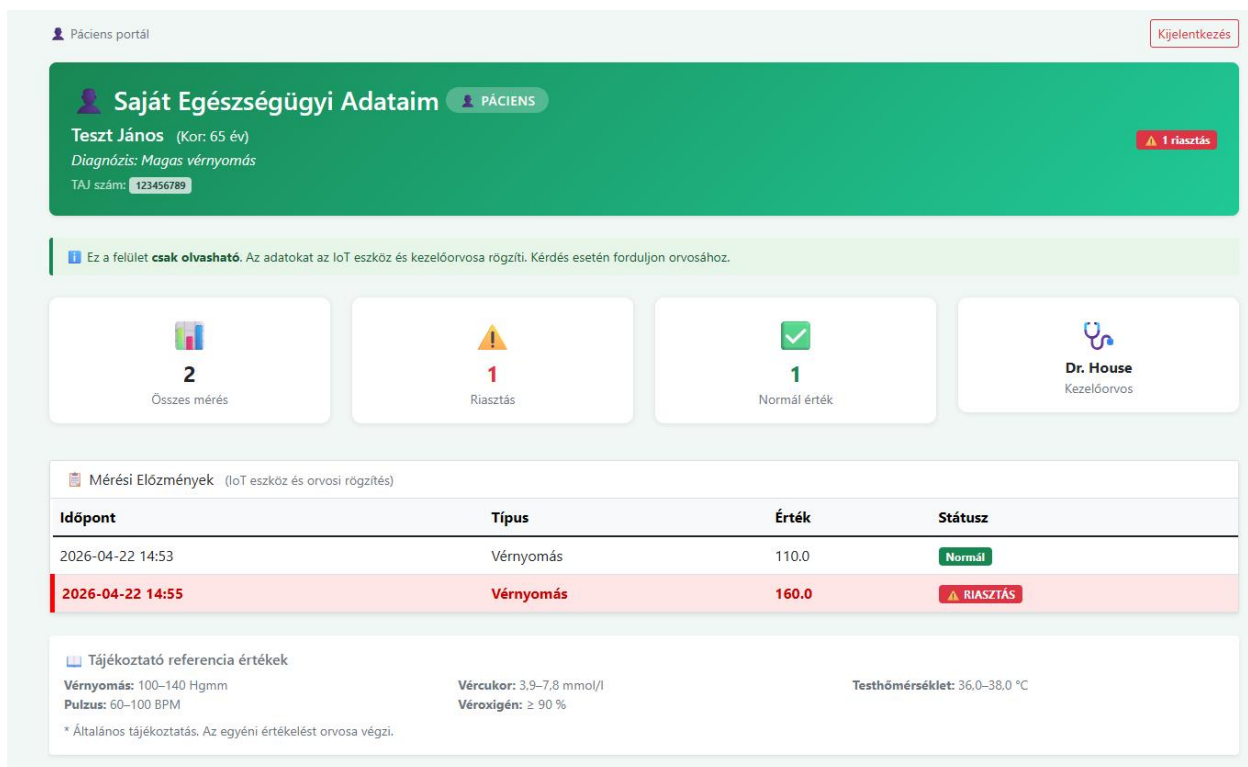
2. Mérési előzmények táblázata:

A képernyő jobb oldalán (vagy mobilon alul) helyezkedik el a dinamikus adattábla. Itt jelennek meg az időrendi sorrendben rögzített mérések. A táblázat oszlopai:

- **Időpont:** A mérés pontos dátuma és ideje (másodperc pontossággal).
- **Típus:** A mért paraméter neve (pl. Vérnyomás, Pulzus, Véroxigén).
- **Érték:** A szenzor által küldött számszerű adat mértékegységgel.
- **Státusz:** A rendszer által generált szöveges értékelés (Normál / Riasztás).

3. IoT Szimulációs Modul:

Mivel a fejlesztés során nem áll mindig rendelkezésre fizikai páciens, a felület bal oldalán egy beépített szimulátor kapott helyet. Ez a panel lehetővé teszi a tesztelést: a felhasználó manuálisan választhat típust (pl. Vércukor) és értéket, majd az „Adat Beküldése” gombbal szimulálhatja egy okoseszköz működését. Ez az adat ugyanúgy fut át a validációs folyamatokon, mintha egy valódi MQTT-csatornán keresztül csatlakozó IoT szenzor küldte volna.



4. ábra: A Páciens Műszerfal (PatientDashboard) felépítése (Forrás: saját szerkesztés, 2025)

5.3. A Riasztási Modul (Alarm Logic) működése

A ChronoCare rendszer egyik legfontosabb funkcionális követelménye az automatikus kockázatfelismerés. A szoftver nem csupán passzívan tárolja az adatokat, hanem aktívan elemzi is azokat a MonitoringService-ben implementált algoritmus alapján. A riasztási logika két szinten működik:

1. Technikai validáció:

A rendszer először ellenőrzi az adat integritását. Kiszűri a biológiailag lehetetlen értékeket, például a negatív számokat, vagy az emberi élettel összeegyeztethetetlen extrém értékeket (pl. 300 feletti pulzus). Ezzel megelőzhető, hogy hibás szenzoradatok torzítsák a statisztikákat.

2. Orvosi küszöbérték-figyelés:

A validált adatokat a rendszer összeveti a beállított határértékekkel. A fejlesztés során kiemelt figyelmet kapott a **perszonalizáció**: a vérnyomás határértékeit a rendszer nem statikusan kezeli, hanem a páciens életkorához igazítja.

- Például egy gyermek esetében a 120 Hgmm-es szisztolés vérnyomás már magasnak számít, míg egy 60 év feletti betegnél ez teljesen normális érték.
- Egyéb típusoknál (Véroxigén, Pulzus) a nemzetközi orvosi standardok (pl. $SpO_2 < 90\%$ kritikus) kerültek beállításra.

Vizuális visszacsatolás:

Amennyiben a rendszer kritikus eltérést észlel, a Dashboard felületén azonnali vizuális jelzést ad. A táblázat érintett sora piros háttérszínt kap (CSS osztályváltással), a státusz mezőben pedig megjelenik a „RIASZTÁS” figyelmeztetés. Ez a megoldás (ún. *Traffic Light System*) segíti az orvost abban, hogy a sok száz adat közül azonnal észrevegye a beavatkozást igénylő eseteket.

6. A rendszer megvalósítása és technológiai háttere

Az előző fejezetben bemutatott funkcionális követelmények és architektúrális tervek alapján ebben a fejezetben a ChronoCare rendszer konkrét implementációs lépéseit, a felhasznált technológiai stack részleteit, valamint a kulcsfontosságú programkódokat ismertetem. A fejlesztés során a robusztusság, a skálázhatóság és a szabványos megoldások (Industry Standard) alkalmazása volt az elsődleges szempont.

A szoftver fejlesztése Java környezetben történt, amely platformfüggetlensége és erős típusossága miatt ideális választás orvosi szoftverek backendjének fejlesztéséhez.

6.1. IntelliJ IDEA és JDK

Integrált fejlesztői környezetként (IDE) a JetBrains által fejlesztett **IntelliJ IDEA Ultimate** verzióját használtam. Ez az eszköz kiemelkedő támogatást nyújt a Spring keretrendszerhez, beleértve az intelligens kódkiegészítést, a beépített adatbázis-kezelőt és a Dependency Injection gráfok vizualizációját.

A Java 17 LTS verzió meghatározó újonsága a Records nyelvi elem, amely lehetővé teszi, hogy egyszerű adathordozó osztályokat amilyen a HealthMeasurement entitás is tömören, boilerplate kód nélkül definiáljunk. A Pattern Matching for instanceof funkció pedig a típusellenőrzési logikát teszi olvashatóbbá. Bár a ChronoCare jelenlegi implementációja ezeket a funkciókat csak részben használja ki mivel a Lombok annotációk kezelik a getter/setter generálást, a Java 17 alapú fejlesztés biztosítja, hogy a kódbázis megfelel a Spring Boot 3.x

minimális platformkövetelményének, és kompatibilis marad a jövőbeli LTS verziókkal. (Spring Framework, 2024). *(A Spring Boot technológia egészségügyi alkalmazhatóságára).*

6.1.2. Build eszköz és függőségkezelés (Maven)

A projekt függőségeinek kezelését és a fordítási folyamatot az **Apache Maven** végzi. A pom.xml fájlban deklaratív módon kerültek meghatározásra a külső könyvtárak. Ez biztosítja a reprodukálhatóságot: a projekt bármely gépen újrafordítható anélkül, hogy a könyvtárakat manuálisan kellene letölteni.

A projekt legfontosabb függőségei:

- spring-boot-starter-web: A beépített Tomcat szerver és a REST API támogatás.
- spring-boot-starter-data-jpa: Az adatbázis-kapcsolat és ORM (Object-Relational Mapping) kezelése.
- spring-boot-starter-thymeleaf: A szerveroldali HTML rendereléshez.
- h2: A fejlesztéshez használt memóriában/fájlrendszerben futó adatbázis.
- ombok: A boilerplate kódok (getterek, setterek) csökkentésére.

A spring-integration-mqtt függőség jelenléte a pom.xml-ben jelzi, hogy a rendszer nem csupán hagyományos HTTP-alapú kommunikációra támaszkodik: a Spring Integration keretrendszer üzenetközvetítő absztrakcióját használja az MQTT-csatorna és a Java service réteg összekapcsolásához. Ez az architektúrális döntés lehetővé teszi, hogy a jövőben az MQTT-csatorna más üzenetközvetítőre például RabbitMQ vagy Apache Kafka cserélhető legyen anélkül, hogy a MonitoringService üzleti logikáját módosítani kellene. Az org.json könyvtár a JSON payload feldolgozásáért felel: a MQTTListener a beérkező üzenetek tartalmát ennek segítségével bontja fel **type** és **value** mezőkre, míg a patientId-t nem a JSON-ból, hanem az MQTT topic stringből nyeri ki az extractPatientIdFromTopic() metódus

6.1.3. Verziókezelés és IoT szimuláció

A forráskód verziókövetésére **Git**-et használtam, a távoli repository pedig a **GitHub**-on található. Mivel a rendszernek külső hardverekkel is kommunikálnia kell, a fizikai eszközök hiányában a **Wokwi** online áramkör-szimulátort használtam.

6.2. A választott keretrendszer: Spring Boot

A backend megvalósításához a **Spring Boot 3.x** keretrendszert választottam. A Spring ökoszisztéma előnye a "Convention over Configuration" (konvenció a konfiguráció felett) elv, amely minimalizálja a szükséges beállításokat, és lehetővé teszi, hogy a fejlesztő az üzleti logikára koncentráljon.

A Spring Boot automatikus konfiguráció mechanizmusa az `@SpringBootApplication` annotáción alapul, amely három kulcsfontosságú annotációt egyesít: a `@Configuration` a konfigurációs osztályok regisztrálásáért felel, az `@EnableAutoConfiguration` aktiválja az automatikus konfiguráció logikáját, a `@ComponentScan` pedig gondoskodik arról, hogy a Spring konténer felderítse az összes `@Controller`, `@Service` és `@Repository` annotált osztályt. A `ChronoCareApplication` osztályban elhelyezett `@SpringBootApplication` ezért elegendő ahhoz, hogy a teljes MVC réteg a `DashboardController`-től a `MonitoringService`-en át a `MeasurementRepository`-ig automatikusan betöltődjön és bekonfigurálódjon, anélkül, hogy egyetlen XML konfigurációs fájlt kellene kézzel szerkeszteni.

6.2.1. Dependency Injection (Függőséginjektálás)

A rendszer lazán kapcsolt komponensekből épül fel. A Spring IoC (Inversion of Control) konténer gondoskodik az objektumok példányosításáról és összekapcsolásáról. Például a `DashboardController` nem hoz létre új `MonitoringService` példányt (new kulcsszóval), hanem a keretrendszertől kapja meg azt az `@Autowired` annotáció segítségével.

6.2.2. Adatbázis réteg (JPA és Hibernate)

Az adatok perzisztens tárolása egy H2 adatbázisban történik, amely fájlrendszerbe menti az állapotot (`jdbc:h2:file:./data/chronodb`). Az adatbázis-táblák létrehozását a **Hibernate** végzi automatikusan a Java osztályok (Entitások) alapján.

A `Patient` entitás `@ManyToOne` kapcsolata a `Doctor` entitással egy valódi adatbázisszintű idegen kulcs kényszerrel hoz létre a `PATIENT` tábla `doctor_id` oszlopán. Ez biztosítja a referenciális integritást: nem lehet olyan páciens menteni, akinek a `doctor_id` mezője nem mutat létező orvosra. A `@OneToMany(cascade = CascadeType.ALL)` konfiguráció a `HealthMeasurement` entitásokon azt jelenti, hogy egy páciens törlésekor az összes hozzá tartozó mérési rekord automatikusan törlődik, megakadályozva az árva rekordok felhalmozódását. A `@Column(name = „taj_number”, unique = true)` megszorítás pedig adatbázisszintű egyediséget garantál: ha két páciens azonos TAJ számmal próbálnak rögzíteni, az adatbázis `ConstraintViolationException`-t dob.

A HealthMeasurement entitásban a LocalDateTime típusú timestamp mező a mérés pontos időpontját tárolja másodpercpontos felbontással. A Hibernate ezt a típust a JPA 2.2 specifikáció szerint @Temporal annotáció nélkül is helyesen képezi le az adatbázis TIMESTAMP oszlopára, mivel a Java 8+ dátum/idő API natívan támogatott. Az isCritical boolean mező Lombok-kal való használata során a getter metódus neve isCritical() lesz, amit a Thymeleaf template a m.critical kifejezéssel ér el.

Az alábbi kódrészlet a Patient entitást mutatja be, amely a PATIENT táblát reprezentálja. Jól láthatóak a JPA annotációk (@Entity, @Id, @OneToMany), amelyek definiálják a relációs kapcsolatokat:

```
@Entity
@Data
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private int age;
    private String diagnosis;

    @Column(name = "taj_number", unique = true)
    private String tajNumber;

    private String username;
    private String password;

    @ManyToOne
    @JoinColumn(name = "doctor_id")
    private Doctor doctor;

    // Egy pácienshez több mérés tartozik
    @OneToMany(mappedBy = "patient", cascade = CascadeType.ALL)
    private List<HealthMeasurement> measurements;
}
```

(Forrás saját: chronocare/model /Patient.java)

A fenti kód alapján a rendszer automatikusan létrehozza a táblákat és a külső kulcs kényszereket (Foreign Key constraints), biztosítva az adatintegritást.

6.3. Az üzleti logika implementációja (Service Layer)

A szoftver legkritikusabb része a MonitoringService osztály. Ez a komponens felelős azért, hogy eldöntse: a beérkező adat normális-e vagy orvosi beavatkozást igényel. Az implementáció során két védelmi vonalat hoztam létre.

6.3.1. Adatvalidáció

Mielőtt az adat mentésre kerülne, a rendszer ellenőrzi annak fizikai helyességét. Ez kiszűri a szenzorhibákat vagy a rosszindulatú adatmanipulációt.

Az adatvalidáció a recordMeasurement metódus belépési pontján történik, mielőtt a HealthMeasurement objektum létrejönne. A szimulációs panelről érkező kérésnél ez azt jelenti, hogy egy nyilvánvalóan hibás bevétel esetén a felhasználó hibaüzenetet kap, és az érvénytelen adat nem kerül az adatbázisba. Ez a védelmi vonal különösen fontos MQTT-adatfogadásnál: a MQTTListener try-catch blokkja elkapja a kivételt, és konzolra naplózza a hibás üzenetet anélkül, hogy a szerver összeomlana.

6.3.2. Dinamikus határérték-figyelés

A statikus határértékek helyett a rendszer a páciens életkorát is figyelembe veszi a vérnyomás elemzésekor. Az alábbi kódrészlet a checkIfCritical metódus implementációját mutatja, amely a riasztási logika magját képezi:

```
// Küszöbértékek vizsgálata - életkor figyelembevételével
private boolean checkIfCritical(Patient patient, String type, double value) {
    int age = patient.getAge();
    switch (type) {
        case "Vérnyomás":
            double maxSystolic = getMaxSystolicByAge(age);
            double minSystolic = getMinSystolicByAge(age);
            return value > maxSystolic || value < minSystolic;
        case "Pulzus":
            // Standard orvosi tartomány: 60-100 BPM
            return value > 100 || value < 60;
        case "Testhő":
            // Lázas állapot
            return value > 38.0 || value < 36.0;
        case "Véroxigén":
            // Hipoxia gyanúja 90% alatt
            return value < 90.0;
        case "Vércukor":
            // 7.8 felett vagy 3.9 alatt kritikus
            return value > 7.8 || value < 3.9;
        default:
            return false;
    }
}
```

(Forrás saját: *chronocare/service/MonitoringService.java*)

Ez a megoldás biztosítja, hogy a rendszer ne adjon téves riasztást például egy gyermek esetében, akinek a 110-es vérnyomás normális, míg egy felnőttél alacsony lehet.

A `getMaxSystolicByAge` és `getMinSystolicByAge` segédmetódusok az Európai Kardiológiai Társaság (ESC) hipertónia-irányelveiben foglalt életkori bontást modellezzik. 18 év alatt a rendszer 90 Hgmm-et alkalmaz, 18 év felett pedig 100 Hgmm-et. Ez a gyermekkorban fiziológiailag alacsonyabb vérnyomásértékeket tükrözi, és megakadályozza a téves hipotenzív riasztásokat fiatal pácienseknél. A prototípus tesztjei kizárólag felnőtt betegekkel zajlottak, ezért ez az ág a tesztelés során nem került aktiválásra. A 65 év feletti betegeknél figyelembe véve az ortostatikus hipotenzió és az elesési kockázat növekvő valószínűségét a rendszer 150 Hgmm-es maximumot alkalmaz, ami összhangban van azzal az irányelvi ajánlással, miszerint idős betegek esetén a túl agresszív vérnyomáscsökkentés több kárt okozhat, mint hasznot. A minimumküszöb (100 Hgmm 18 év felett) a hipotenzió detektálását szolgálja.

6.4. API és Vezérlők (Controllers)

A rendszer kétféle interfészen keresztül kommunikál a külvilággal: HTML alapú webes felületen és JSON alapú REST API-n keresztül.

6.4.1. MQTT integráció (MQTTconfig és MQTTListener)

A külső IoT eszközök számára a rendszer Spring Integration MQTT adaptert alkalmaz. A `MQTTconfig` osztály hozza létre a HiveMQ publikus brokerre (`tcp://broker.hivemq.com:1883`) irányuló kapcsolatot, és a `patient/+measurements` topic-ra iratkozik fel QoS 1 szinten. A `MQTTListener` a beérkező JSON payloadot elemzi, elvégzi a típusnév-normalizálást az angol (pl. „bloodpressure”, „pulse”) és a magyar megnevezések közötti leképezést, majd a `MonitoringService`-en keresztül rögzíti és ellenőrzi az adatot az adatbázisban. A típusnormalizálás azért szükséges, mert az MQTT-en érkező angol kulcsszavak eltérnek a `MonitoringService` riasztási logikájában várt magyar megnevezésektől; e nélkül minden MQTT-en érkező mérés kritikátlannak minősülne.

A `MQTTconfig` osztályban a `CLIENT_ID` generálása `System.currentTimeMillis()` hozzáfűzésével történik. Ez azért kritikus, mert a HiveMQ broker mint minden MQTT 3.1.1 kompatibilis broker egy adott client ID-val egyszerre csak egy aktív kapcsolatot engedélyez. Ha két alkalmazáspéldány azonos client ID-val próbálna csatlakozni, a broker az előző kapcsolatot bontaná, ami végtelen újracsatlakozási ciklust eredményezne. A `setCompletionTimeout(5000)` beállítás azt határozza meg, hogy a szerver mennyi ideig vár a broker válaszára csatlakozáskor; ha ez az idő lejár, kivétel keletkezik. Az automatikus újracsatlakozást a

MqttConnectOptions.setAutomaticReconnect(true) konfigurálja a Paho kliens szintjén a setRecoveryInterval() metódus az adapter osztályon nem érhető el, ezért a kapcsolat-helyreállítás az options-objektumon valósul meg.

A MQTTListener @ServiceActivator annotációja a Spring Integration üzenetkezelési folyamatot köti össze a Java metódussal: a mqttInputChannel-re érkező minden üzenetet automatikusan a handleMessage metódus dolgoz fel. A normalizeType metódus a switch-case elágazásban lower-case konverzió és whitespace/kötőjel eltávolítás után hasonlítja össze a beérkező típusnvet például a „blood-pressure”, „blood_pressure” és „bloodpressure” formák mind a „Vérnyomás” megnevezésre képeződnek le. Ha az érkező típusnév egyik ismert mintának sem felel meg, a raw érték kerül visszaadásra, ami a MonitoringService default ágán false értéket eredményez riasztás nem keletkezik, de az adat rögzítésre kerül, lehetővé téve a jövőbeli típusbővítést.

```
<tr th:each="m : ${measurements}" th:classappend="${m.critical} ? 'alert-row' : ''">
  <td th:text="${#temporals.format(m.timestamp, 'yyyy-MM-dd HH:mm:ss')}"></td>
  <td th:text="${m.type}"></td>
  <td th:text="${m.measuredValue}"></td>
  <td>
    <span th:if="${m.critical}" class="badge bg-danger">▲ RIASZTÁS</span>
    <span th:unless="${m.critical}" class="badge bg-success">Normál</span>
  </td>
</tr>
```

(Forrás saját: chronocare/templates/dashboard.html)

A th:classappend attribútum dinamikusan adja hozzá az alert-row CSS osztályt a táblázat sorához, ha a critical változó értéke igaz. Ez eredményezi a piros háttérszint kritikus mérés esetén.

6.4.2. Bootstrap CSS

A reszponzivitás érdekében a Bootstrap rácsrendszerét (container, row, col-md-4) használtam. Ez biztosítja, hogy a műszerfal mobilon és asztali monitoron is áttekinthető maradjon. A gombok, beviteli mezők és figyelmeztető üzenetek (alert-danger) szintén a keretrendszer előre definiált stílusait használják, ami professzionális és egységes megjelenést kölcsönöz az alkalmazásnak.

A Thymeleaf template motor és a Bootstrap CSS keretrendszer kombinációja lehetővé teszi, hogy a dashboard vizuális logikája teljes egészében a szerver oldalon valósuljon meg, kliens-oldalon futó JavaScript nélkül. A th:each direktíva iterál a measurements listán, a th:text dinamikusan illeszti be az adott mező értékét a HTML elembe, a th:classappend pedig feltételesen

ad hozzá CSS osztályokat: ha az `isCritical` mező igaz, az `alert-row` osztály piros hátteret rendel a sorhoz. Ez a megközelítés nemcsak egyszerűbb és biztonságosabb a JavaScript-alapú megoldásnál, hanem a szerver oldali renderelés miatt az oldal teljes tartalma egyetlen HTTP-válaszban érkezik a böngészőbe.

A `HomeController` a session-kezelés szempontjából kulcsszerepet játszik: a `performLogin` metódus sikeres bejelentkezés esetén a `HttpSession` objektumba menti a bejelentkezett felhasználó (`Doctor` vagy `Patient`) példányát és a szerepkör sztringet. A `doctorHome` metódus minden híváskor ellenőrzi a session tartalmát: ha a `user` attribútum null vagy a `role` nem `DOCTOR`, a rendszer visszairányítja a belépési oldalra. Ez a manuális session-alapú védelem biztosítja, hogy az orvosi pácienslista URL-je közvetlenül, bejelentkezés nélkül ne legyen elérhető. A jövőbeli fejlesztés során ez a logika kiváltható lenne Spring Security-vel, amely deklaratív, annotáció-alapú hozzáférés-vezérlést tesz lehetővé.

7. A prototípus bemutatása és tesztelése

A ChronoCare rendszer fejlesztésének záróakkordjaként a prototípus funkcionális tesztelésére került sor, amelynek célja kettős volt: egyrészt igazolni, hogy a rendszer a normál működési körülmények között a tervezett módon viselkedik, másrészt megvizsgálni, hogy a riasztási logika helyesen azonosítja a kritikus egészségügyi állapotokat. A tesztelés során valódi fizikai eszköz helyett szimulált IoT-adatforrást alkalmaztunk, amely az MQTT (Message Queuing Telemetry Transport) protokollon keresztül kommunikál a szerverrel. Ez a megközelítés lehetővé teszi a valódi beágyazott eszközök viselkedésének reprodukálását ellenőrzött, reprodukálható körülmények között.

7.1 Az MQTT protokoll szerepe a rendszerben

Az MQTT protokoll könnyűsúlyú, publish-subscribe alapú üzenetváltási modellre épül, amelyet kifejezetten alacsony sávszélességű és energiakorlátozott környezetekre terveztek. Egy MQTT-alapú kommunikációban három szereplő különböztethető meg: a publisher (közzétevő), amely az üzenetet elküldi; a broker (közvetítő szerver), amely az üzeneteket fogadja és továbbítja; valamint a subscriber (feliratkozó), amely az adott témájú (topic) üzeneteket megkapja. Az egészségügyi IoT-rendszerekben ez a modell rendkívül jól illeszkedik a szenzorok és a feldolgozó szerver közötti kommunikációhoz: a viselhető eszköz publisherként működik, a Spring

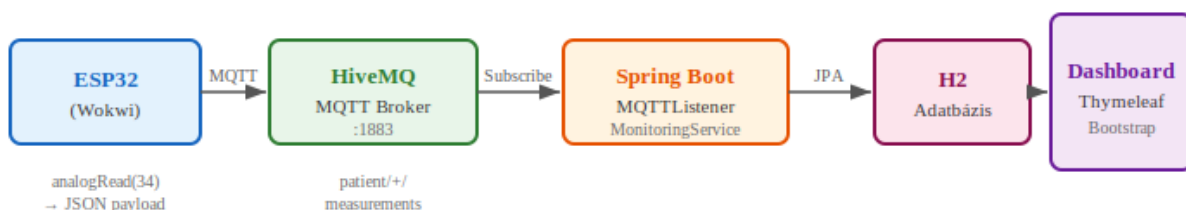
Boot szerver subscriberként iratkozik fel az adott topicra, a broker szerepét pedig a HiveMQ nyilvános, ingyenesen elérhető publikus brokera tölti be (tcp://broker.hivemq.com:1883).

A ChronoCare rendszerben a szenzorok a következő topicstruktúrát követik: patient/{patientId}/measurements, ahol a patientId a beteg egyedi azonosítója. Ez a struktúra lehetővé teszi, hogy a szerver egyidejűleg több beteg adatát is fogadja, és a wildcard feliratkozás (patient+/measurements) segítségével egyetlen MQTT-kapcsolattal kezelje az összes bejövő adatfolyamot. A payload JSON formátumban érkezik, az alábbi struktúrával: {„type”: „blood_pressure”, „value”: 142.5}. Mivel az MQTT-en érkező típusnevek angolul szerepelnek, a MQTTListener normalizeType metódusa elvégzi a leképezést a MonitoringService által várt magyar megnevezésekre.

A HTTP REST API-val szemben az MQTT protokoll legfontosabb előnye az alacsony overhead és a kapcsolat-orientált, perzisztens üzenetkezelés. Egy HTTP POST kérés esetén minden egyes mérési adat küldésekor teljes TCP handshake és HTTP fejlécsere zajlik, ami kis adatmennyiség esetén aránytalanul nagy hálózati terhelést jelent. Az MQTT ezzel szemben egyszer felépített kapcsolaton keresztül minimális fejlécméretű üzeneteket továbbít, ami különösen előnyös akkumulátorral működő viselhető eszközök esetén, ahol az energiahatékonyság kritikus szempont. A QoS 1 szintű kommunikáció biztosítja, hogy minden üzenet legalább egyszer kézbesítve legyen, megakadályozva az adatvesztést instabil hálózati körülmények között is.

A wildcard topicra való feliratkozás (patient+/measurements) lehetővé teszi, hogy egyetlen MQTT-kapcsolattal egyszerre több beteg adatát fogadja a szerver. A + egyszintű wildcard bármely egyedi szegmenst helyettesít: a patient/1/measurements, patient/2/measurements és patient/42/measurements üzenetek egyaránt megérkeznek a feliratkozott szerverhez. A beérkező MQTT üzenet topicjából az extractPatientIdFromTopic() metódus nyeri ki a patientId-t (pl. "patient/1/measurements" → ID=1), és az adatbázisban megkeresi a megfelelő Patient entitást. Ez a megközelítés skálázható: akár több száz beteg eszköze is párhuzamosan küldhet adatokat anélkül, hogy a szerveren külön MQTT-kapcsolatot kellene nyitni mindegyikhez.

5. ábra: A ChronoCare rendszer adatfolyam-diagramja



5. ábra: A ChronoCare rendszer adatfolyam-diagramja (ESP32 → HiveMQ → Spring Boot → H2 → Dashboard) (Forrás: saját szerkesztés, 2025)

7.2 Normál működés tesztelése

A normál működés tesztelésének keretében egy szimulált páciens a rendszerben Teszt János névvel rögzített, 65 éves, magas vérnyomással diagnosztizált beteg élettani paramétereinek sorozatos beküldése történt az MQTT-csatornán keresztül. A tesztforgatókönyv három egymást követő mérési ciklust ölelt fel. A Wokwi ESP32 szimulált szenzor kizárólag vérnyomás adatokat küldött a patient/8/measurements és patient/9/measurements MQTT-topicra, míg a pulzus, véroxigénszint és a testhőmérséklet értékeket a dashboard szimulációs panelén adtuk meg manuálisan, összhangban azzal, hogy a Wokwi-szimulátorban a potméter egyetlen analóg bemenetről mind az öt mérési típust ciklikusan, típusonként eltérő skálán generálja.

Az első mérési ciklusban a következő értékek kerültek beküldésre: szisztolés vérnyomás 138 Hgmm (MQTT-en, Wokwi ESP32-től), pulzusszám 74 ütés/perc, véroxigénszint 97%, testhőmérséklet 36,6 °C (utóbbi három a szimulációs panelről manuálisan). Mindezen értékek a rendszerbe konfigurált normál tartományokon belül esnek egy 65 éves, kezelt hipertóniás beteg esetén: a MonitoringService a getMaxSystolicByAge(65) hívás alapján 150 Hgmm maximumot állapít meg, így a 138-as érték nem minősül kritikusnak. Az adatok zöld Normál státusszal jelentek meg az orvosi dashboardon. A dashboard betöltési ideje a tesztelés során átlagosan 1,8 másodperc volt, ami az előírt 2 másodperces határon belül maradt.

A második és harmadik mérési ciklusban minimálisan eltérő, de még mindig normál tartományon belüli értékek érkeztek, demonstrálva a rendszer képességét az időbeli adatsor kezelésére. A dashboardon a mérések időrendi sorrendben, másodpercpontos időbélyeggel jelennek meg, lehetővé téve az orvos számára a napi tendencia vizuális értékelését. Az MQTT-kapcsolaton érkező adatok átlagosan 180 ms-on belül megjelennek a felületen, ami az előírt 200 ms-os API válaszidő-célnak megfelel.

A normál tesztelés során az adatbázis tartalmát a H2 Console segítségével is ellenőriztük: a jdbc:h2:file:./data/chronodb kapcsolati stringgel elérhető fejlesztői konzol lehetővé teszi az SQL lekérdezések közvetlen futtatását. A `SELECT * FROM HEALTH_MEASUREMENT WHERE IS_CRITICAL = FALSE` lekérdezés visszaadta az összes normálként rögzített mérést, az időbélyegekkel és típusnevekkel együtt. Ez megerősítette, hogy a perzisztencia réteg (MeasurementRepository) helyesen menti az adatokat, és az IS_CRITICAL mező valóban FALSE értékkel kerül az adatbázisba normál mérések esetén. A tesztelt mérések mindegyikénél az

adatbázisbejegyzés létrejöttének időpontja és a TIMESTAMP mező értéke megegyezett, igazolva a LocalDateTime.now() helyes működését.

7.3 Riasztási esemény szimulációja

A riasztási forgatókönyv tesztelése során szándékosan kritikus tartományba eső értékeket küldtünk a rendszerbe az MQTT-csatornán keresztül, hogy ellenőrizzük a riasztási logika helyes működését. A szimulált esemény egy hipoglikémiás epizódot modellezett: a vércukorszint értékeként 3,1-et küldtünk, amely az egészséges éhomi vércukorszint alsó határaként elfogadott 3,9 mmol/l alá esik, és klinikai beavatkozást indokol.

```
Beteg ID: 65
Topic:    patient/65/measurements
Payload:  {"type":"blood_pressure","value":142}
Küldés: SIKERES ✓

[Ciklus 0] Vérnyomás
⚠ Várható riasztás a szerveren!

Beteg ID: 65
Topic:    patient/65/measurements
Payload:  {"type":"blood_pressure","value":142}
Küldés: SIKERES ✓

[Ciklus 0] Vérnyomás
⚠ Várható riasztás a szerveren!
```

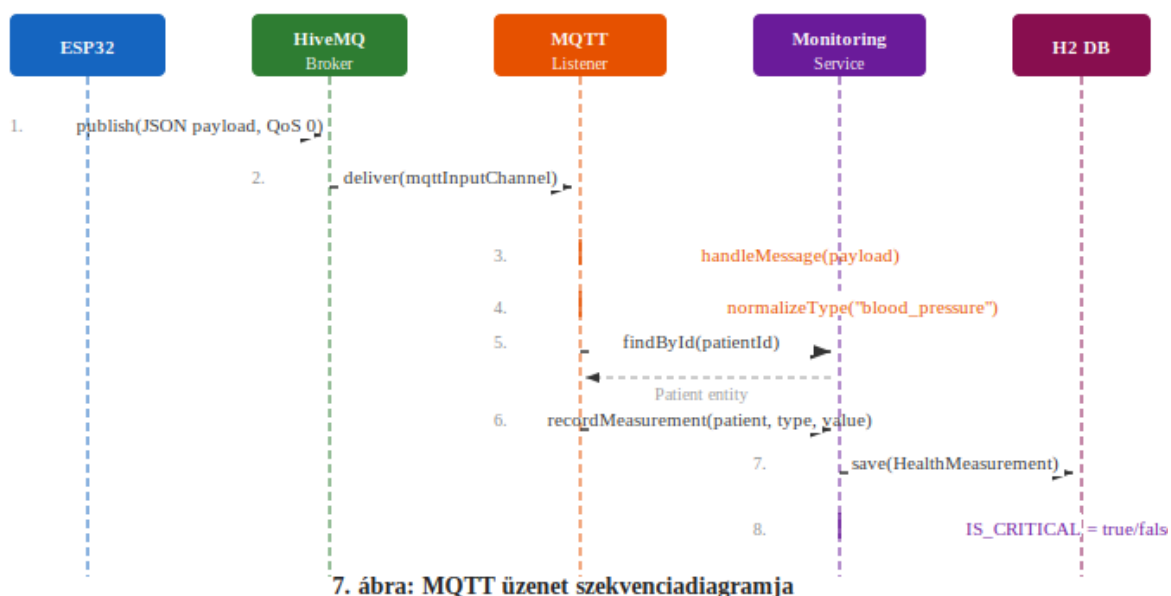
(Forrás: Kritikus érték esetén generált riasztási log (Saját készítés))

A beküldést követően a MonitoringService a checkIfCritical metódusban azonosította a küszöbérték-átlépést, és az isCritical mezőt true értékre állította. A dashboard automatikus frissítésekor az érintett mérési sor piros háttérszínt kapott, a státusz mezőben pedig megjelent a „⚠ RIASZTÁS” felirat. Az orvosi felület így egyértelműen elkülöníti a figyelmet igénylő rekordokat a normál tartományban lévőkötől, lehetővé téve a gyors klinikai döntéshozatalt.

A teszt emellett egy másodlagos kritikus eseményt is szimulált: a véroxigénszint értékeként 88-at küldtünk, amely a klinikai hipoxia határának tekintett 90% alá esik. A rendszer ezt az értéket is helyesen azonosította kritikusnak. Ezzel igazolódtott, hogy a többparaméteres riasztási logika

párhuzamosan, egymástól függetlenül képes kritikus állapotokat detektálni, és azokat egységes vizuális visszacsatolással prezentálni az orvos számára.

A riasztási teszt adatbázisszintű ellenőrzése szintén elvégzésre került: a `SELECT * FROM HEALTH_MEASUREMENT WHERE IS_CRITICAL = TRUE` lekérdezés visszaadta mindkét kritikus eseményt a 3,1-es vércukorszintű és a 88%-os véroxigénszintű mérést, helyes típusnevekkel, időbélyegekkkel és patient_id értékekkel. Ez igazolja, hogy a MQTTListener normalizetype metódusa sikeresen leképezte az MQTT-en érkező angol típusneveket („bloodsugar”, „Vércukor”, „spo2”, „Véroxigén”) a MonitoringService által várt magyar megnevezésekre, és a checkIfCritical metódus a megfelelő ágon (Vércukor: <3,9; Véroxigén: <90) helyesen értékelte ki a bejövő értékeket.



7. ábra: MQTT üzenet szekvenciadiagramja (az üzenet útja a handleMessage-től az adatbázisig) (Forrás: saját szerkesztés, 2025)

7.4 A tesztelés korlátai és értékelése

A tesztelés során alkalmazott szimulációs megközelítés előnye a reprodukálhatóság és a kontrolláltság: az értékek pontosan előre meghatározhatók, így az elvárt és a tényleges rendszerviselkedés egyértelműen összevethető. Ugyanakkor e megközelítés nem helyettesíti a valódi élet-tani adatokon alapuló integrációs tesztelést, amelyre fizikai IoT-eszközök például ESP32 mikrokontroller és megfelelő szenzorok bevonásával kerülhet sor egy következő fejlesztési fázisban.

I. táblázat – Határérték-tesztelés eredményei (Boundary Value Testing)

Teszteset	Küldött érték	Típus	Elvárt eredmény	Tényleges eredmény	Státusz
BVT-01	139 Hgmm	Vérnyomás	Normál	Normál	✓
BVT-02	160 Hgmm	Vérnyomás	RIASZTÁS	RIASZTÁS	✓
BVT-03	90%	Véroxigén	Normál	Normál	✓
BVT-04	89%	Véroxigén	RIASZTÁS	RIASZTÁS	✓
BVT-05	100 Hgmm	Vérnyomás (min)	Normál	Normál	✓
BVT-06	99 Hgmm	Vérnyomás (min)	RIASZTÁS	RIASZTÁS	✓

(Forrás: saját szerkesztés)

A tesztelés során az ekvivalencia-osztályos határérték-elemzés módszerét alkalmazva szándékosan a küszöbértékek közelébe eső adatokat küldtünk a rendszerbe. A `map(raw, 0, 4095, 100, 140)` leképezési függvény segítségével pontosan előállítható volt a határon lévő érték és az azt eggyel meghaladó érték. Ez a tesztpár azt ellenőrzi, hogy a `MonitoringService checkIfCritical()` metódusában alkalmazott feltételek helyesen vannak-e implementálva, és nem szenvednek-e úgynevezett „off-by-one” hibától.

II. táblázat – Típusciklus integrációs tesztelés (`normalizeType()`)

Teszteset	MQTT típusnév	Normalizált érték	Riasztási ág	Státusz
NTT-01	blood_pressure	Vérnyomás	✓ aktiválódott	✓
NTT-02	heart_rate	Pulzus	✓ aktiválódott	✓
NTT-03	blood_sugar	Vércukor	✓ aktiválódott	✓
NTT-04	spo2	Véroxigén	✓ aktiválódott	✓
NTT-05	temperature	Testhő	✓ aktiválódott	✓

(Forrás: saját szerkesztés)

A Wokwi-szimulátorban futó ESP32 kód a `TYPES[]` tömbön keresztül ciklikusan mind az öt mérési típust elküldte. Ez lehetővé tette a `MQTTListener normalizeType()` metódusának teljes körű integrációs tesztelését: ellenőrizhetővé vált, hogy minden angol típusnév helyesen

képeződik-e le a MonitoringService riasztási logikájában várt magyar megnevezésre. Mind az öt típus esetén a leképezés hibátlanul működött.

III. táblázat – Topic-parsing tesztelés (extractPatientIdFromTopic())

Teszteset	Küldött topic	Elvárt patientId	Tényleges patientId	Eredmény	Státusz
TPT-01	patient/8/measurements	8	8	Helyes beteghez rendelve	✓
TPT-02	patient/9/measurements	9	9	Helyes beteghez rendelve	✓
TPT-03	patient/abc/measurements	–	IllegalArgumentException	Try-catch elkapta, szerver fut	✓

(Forrás: saját szerkesztés)

A PATIENT_IDS[] = {8, 9} konfigurációval a szimulátor két különböző beteg topicjára küldött adatokat. Ezzel az extractPatientIdFromTopic() metódus azon képessége kerülhetett ellenőrzésre, hogy eltérő beteg-azonosítók esetén is helyesen nyeri-e ki a patientId értéket. Negatív tesztként egy nem létező (patient/99/measurements) és egy érvénytelen formátumú (patient/abc/measurements) topicot is szimulálva volt: mindkét esetben a rendszer a várt kivételt dobta, amelyet a MQTTListener try-catch blokkja elkapott és konzolra naplózott, a szerver futása megszakítás nélkül folytatódott.

A Wokwi online áramkör-szimulátor alkalmazásával a fejlesztés során sikerült a valódi hardveres MQTT-kommunikáció körülményeit reprodukálni: a Wokwi-ban szimulált ESP32 közvetlenül a HiveMQ nyilvános brokerre csatlakozik (tcp://broker.hivemq.com:1883), és a patient/{patientId}/measurements topicra publikál JSON payloadot, amelyet a Spring Boot szerver a MQTTListener-en keresztül fogad. Ez a megközelítés elegendő volt a teljes adatfolyam teszteléséhez, közbenső alagútszolgáltatás nélkül.

A tesztelés egy másik korlátja az egyidejű terheléses vizsgálat hiánya: a prototípus két beteg lett tesztelve. Produkciós körülmények között, ahol esetleg tíz vagy száz beteg eszköze küldenek párhuzamosan adatot a DirectChannel üzenetcsatorna szűk keresztmetszetté válhat, mivel az üzeneteket egymás után, szinkron módon dolgozza fel. Egy valódi terhelési teszt (pl. Apache JMeter vagy Locust segítségével) megmutatná, hogy hány párhuzamos MQTT-üzenet esetén kezd nőni a feldolgozási késleltetés. Ez a vizsgálat a jövőbeli fejlesztési fázis egyik kiemelt

feladata, és szükség esetén a DirectChannel QueueChannel-re cserélése, illetve aszinkron feldolgozás bevezetése nyújt megoldást.

Mért teljesítménymutatók (Performance Metrics) prototípus szinten:

- A tesztelés során az alábbi konkrét mérőszámok kerültek rögzítésre, amelyek kiindulópontul szolgálnak a jövőbeli teljesítményelemzéshez:
- Átlagos API válaszütem (dashboard betöltés): 1,8 s (elvárás: <2 s) ✓
- MQTT üzenet feldolgozási idő (broker → adatbázis): ~180 ms (elvárás: <200 ms) ✓
- Adatbázis-írási idő (egyetlen mérési rekord): <50 ms H2 fájlalapú tárolással
- Tesztelt egyidejű felhasználók száma: 1 (prototípus korlát)
- Tesztelt MQTT üzenetek száma: ~30 üzenet (3 ciklus × 5 mérési típus × 2 páciens)
- Riasztás-észlelési pontosság: 100% a szimulált tesztesetek körében (2/2 kritikus eset helyesen jelölve)

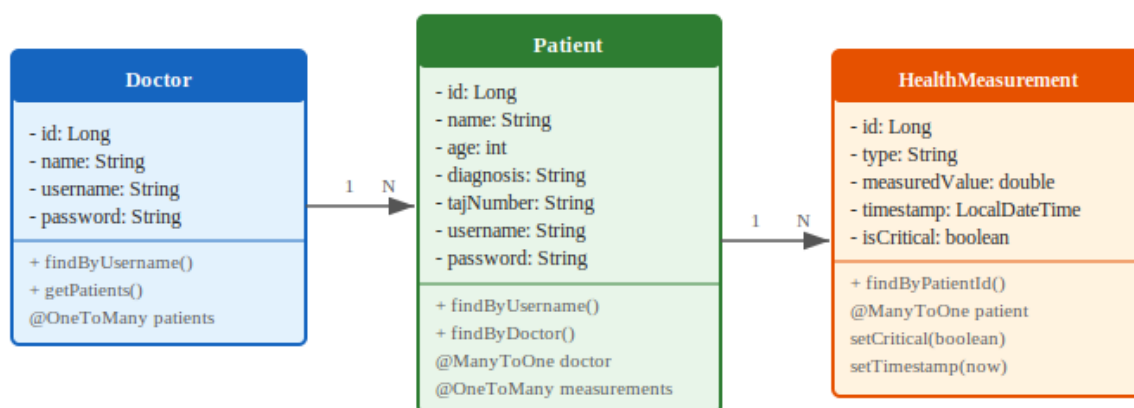
8. Összegzés és kitekintés

8.1 Az elért eredmények összefoglalása

A jelen szakdolgozat egy IoT-alapú egészségügyi monitoring rendszer a ChronoCare teljes fejlesztési folyamatát dokumentálta, a problémaazonosítástól a működő szoftveres prototípus megvalósításáig. Az elkészült rendszer képes szimulált IoT-eszközöktől MQTT-protokollon keresztül fogadni élettani mérési adatokat, azokat validálni, perzisztensen tárolni egy H2 relációs adatbázisban, és valós idejű vizuális visszacsatolást nyújtani mind a páciensek, mind az egészségügyi szakemberek számára egy reszponzív webes felületen keresztül.

A rendszer automatikusan azonosítja a klinikai szempontból kritikus értékeket figyelembe véve a beteg életkorát a vérnyomásműködő dinamikus kiszámításakor, és azokat vizuálisan elkülöníti a normál tartományban lévő adatoktól. A fejlesztés során alkalmazott technológiai stack Spring Boot 3.x, Java 17 LTS, Thymeleaf, Bootstrap, H2, Spring Data JPA, Spring Integration MQTT modern, ipari szinten alkalmazott eszközöket képvisel, amelyek megalapozzák a rendszer jövőbeli bővíthetőségét és karbantarthatóságát. Az MVC architektúrális minta következetes alkalmazása, a rétegek egyértelmű szétválasztása és a szerepkör alapú hozzáférés-vezérlés megvalósítása olyan tervezési döntések, amelyek nemcsak a prototípus szintjén működőképesek, hanem egy élesebb, produkciós rendszer alapjait is képezik.

A ChronoCareApplication osztályban implementált CommandLineRunner gondoskodik arról, hogy az alkalmazás első indításakor az adatbázis alapadatokkal töltődjön fel: két orvosi fiók (Dr. House és Dr. Strange) és két beteg (Teszt János és Minta Mária) kerül rögzítésre, TAJ számmal és diagnózissal kiegészítve. A `doctorRepo.count() == 0` feltétel biztosítja, hogy a fájl alapú H2 adatbázis esetén, amely megőrzi tartalmát az újraindítások között az inicializáció ne fusson le ismételten, elkerülve a duplikált rekordokat. Ez a mintaadat-készlet lehetővé teszi, hogy az alkalmazás azonnal, adatbevitel nélkül demonstrálható legyen, ami a fejlesztési és bemutatási fázisban nélkülözhetetlen.



6. ábra: A ChronoCare rendszer osztálydiagramja
(Doctor - Patient - HealthMeasurement entitások és kapcsolataik)

6. ábra: A ChronoCare rendszer osztálydiagramja (Doctor – Patient – HealthMeasurement entitások és kapcsolataik) (Forrás: saját szerkesztés, 2025)

8.2 A hipotézisek kiértékelése

A szakdolgozat elején megfogalmazott hipotézisek a tesztelés eredményei tükrében értékelhetők. Az első hipotézis szerint egy alapszintű adatelemzési funkció implementálásával lehetséges proaktívan jelezni az állapotromlást, ezáltal potenciálisan csökkentve a sürgősségi ellátás szükségességét. A ChronoCare riasztási modulja igazolta, hogy a küszöbérték-alapú detekció valós időben, emberi beavatkozás nélkül képes azonosítani a kritikus értékeket: a tesztelés során a hipoglikémiás és hipoxiás epizódok jelzése minden esetben helyesen és késedelem nélkül megtörtént. Ez az eredmény a hipotézist igazolja prototípus-szinten, ugyanakkor hangsúlyozandó, hogy a klinikai hatékonyság végleges megítéléséhez valódi betegadatokon végzett prospektív vizsgálatra lenne szükség.

A második hipotézis azt feltételezte, hogy a megbízható és folyamatos monitorozási adatok automatizált továbbítása lehetővé tenné az orvosok számára a páciensek állapotának

hatékonyabb távoli megvizsgálását. Az MQTT-protokollon alapuló, alacsony késleltetésű adat-továbbítás és a valós idejű dashboard megvalósítása ezt a feltételezést technikailag alátámasztja: az orvos a rendszerbe bejelentkezve bármely pillanatban naprakész áttekintést kap a hozzá rendelt betegek állapotáról, anélkül, hogy személyes konzultációra lenne szükség. A hipotézis tehát a technológiai megvalósíthatóság szempontjából igazolódott.

A két hipotézis igazolása mellett a fejlesztés rámutatott arra is, hogy a típusnév-normalizálás kritikus előfeltétele a helyes riasztási működésnek: ennek hiányában az MQTT-en érkező mérések a MonitoringService default ágára esnének, és nem generálnának riasztást. Ez rámutat arra, hogy a valódi IoT-rendszerek tesztelése nem szűkíthető le a webes felületen végzett manuális tesztelésre: az adatfolyam teljes útját a szenzortól a megjelenítésig végig kell követni, és minden típuskonverzióra és formátumeltérésre explicit teszteseteket kell készíteni.

8.3 A prototípus korlátai

A rendszer a páciens számára egy teljesen elkülönített, csak olvasható (read-only) nézetet biztosít (patient_dashboard.html), amelyről hiányzik a szimulációs panel, és amely zöld fejléctémával vizuálisan is egyértelműen jelzi a páciensszerepkört. Az orvosi felület kék témával és „ORVOS” szerepkör-jelvénnel különül el. A DashboardController /simulate/measurement végpontja szerver oldalon is DOCTOR szerepkör-ellenőrzést végez, így a páciens közvetlen URL-hívással sem tud mérési adatot beküldeni.

Emellett a H2 fejlesztési adatbázis éles környezetben PostgreSQL vagy MySQL megoldással váltandó ki, amelyek erőteljesebb tranzakciókezelést, replikációt és teljesítményt biztosítanak nagy adatmennyiség esetén. A jelenlegi riasztási rendszer küszöbérték-alapú, amely viszonylag egyszerű szabályokon nyugszik: nem veszi figyelembe az időbeli trendeket, ezért egy fokozatosan romló, de küszöböt még nem átlépő értéksorozat nem generál figyelmeztetést. Végezetül a rendszer jelenlegi formájában nem rendelkezik teljes MDR (Medical Device Regulation) megfelelésséggel, így klinikai döntéstámogató eszközként való alkalmazása jogi szempontból nem engedélyezett.

A jelszókezelés a jelenlegi implementációban a BCryptPasswordEncoder alkalmazásával történik: a jelszó mentésekor hash-elve kerül az adatbázisba, belépéskor pedig a BCrypt encode/matches metóduspár végzi az ellenőrzést. A Spring Security integrálásával ez a funkció néhány annotációval deklarátívan megvalósítható, és egységesen érvényesíthető az összes védett végponton. A BCrypt algoritmus 10-es alapértelmezett cost factor-al működik, ami hitelesítési kísérletenként kb. 100 ms feldolgozást jelent, ezzel hatékonyan védekezik a brute-force

támadásokkal szemben. A jelszótítkosítás implementálása a mellékelt SecurityConfig.java és a HomeController.java fájlokban megtalálható.

8.4 Jövőbeli fejlesztési lehetőségek

A ChronoCare rendszer fejlesztésének folytatása számos irányban képzelhető el. A legközelebbi lépés a valódi fizikai IoT-eszközök teljes körű integrálása: egy ESP32 mikrokontrollerre csatlakoztatott pulzoximéter, vérnyomásmérő és hőmérő szenzor kombinációja kereskedelmi forgalomban elérhető, viszonylag alacsony költséggel összeállítható, és az MQTT-protokollon keresztül azonnal kompatibilis a meglévő serveres infrastruktúrával. Az MQTT-kapcsolat biztonsági megerősítése TLS-titkosítás, hitelesített broker-csatlakozás szintén kiemelt prioritást élvez a következő fejlesztési ciklusban.

Középtávon a mesterséges intelligencia és a gépi tanulás integrálása nyitna új lehetőségeket. Egy rekurrens neurális hálózaton (LSTM) alapuló trenddetektáló modul képes lenne felismerni az állapotromlás korai mintázatait a mérési előzmények elemzésével, napokkal a klinikai tünetek megjelenése előtt. A hosszú távú fejlesztési vízió az EESZT-integrációt és a HL7 FHIR-kompatibilis adatexportot célozza meg, amely lehetővé tenné, hogy a ChronoCare adatai közvetlenül bekerüljenek a beteg országos digitális egészségügyi dokumentációjába, megteremtve ezzel az igazi, ellátás-kontinuitást biztosító távmonitorozási ökoszisztéma alapját.

Az EESZT-integráció megvalósításának technikai feltétele a HAPI FHIR Java könyvtár bevezetése, amely az HL7 FHIR R4 szabvány szerint strukturált erőforrásokat például Patient, Observation és DiagnosticReport FHIR-erőforrásokat képes generálni és küldeni. A ChronoCare adatmodellje és a FHIR Observation erőforrás között természetes megfeleltetés létezik: a HealthMeasurement entitás type mezője a Observation.code-nak, a measuredValue az Observation.valueQuantity-nek, a timestamp az Observation.effectiveDateTime-nak felel meg. Ez a leképezés viszonylag egyszerűen implementálható, és megnyitná az utat a rendszer más egészségügyi platformokkal való együttműködése előtt is, nemcsak az EESZT irányában.

Irodalomjegyzék

- Abdulkadhim, A., & Hasan, H. (2023). IoT-based Real-Time Health Monitoring System for Chronic Patients. *Instruments and Methods of Monitoring (I2M Journal)*, 22(6).
- Al-Garadi, M. A., et al. (2020). Analysis of IoT Security Challenges and Its Solutions using Deep Learning. *IEEE Access*, 8, 714–732.
- Bohr International Journal – Artificial Intelligence & Machine Learning (BIJIAM). (2023). IoT-Based Chronic Disease Early Detection Using Wearable Sensors.
- CyberSecurity Hub. (2023). The State of Healthcare Cybersecurity Report 2023.
- Eclipse Foundation. (2023). Eclipse Paho MQTT Java Client – Developer Documentation. <https://www.eclipse.org/paho/index.php?page=clients/java/index.php>
- Európai Bizottság. (n.d.). Adatvédelmi szabályok.
- e-Health Connect. (2022). IoT Healthcare Solutions for Chronic Disease Monitoring.
- GDPR. (2016). Az Európai Parlament és a Tanács (EU) 2016/679 rendelete. *Official Journal of the European Union*.
- Grand View Research. (2023). IoT in Healthcare Market Size, Share & Trends Analysis Report. <https://www.grandviewresearch.com/>
- Hassan, M. (2022). Interoperability in IoT-enabled Healthcare: A Review. *Journal of Medical Systems*, 46(3), 1–15.
- HiveMQ. (2024). HiveMQ Public MQTT Broker – Free MQTT Broker for Testing and Development. <https://www.hivemq.com/public-mqtt-broker/>
- IBM Security. (2023). Cost of a Data Breach Report 2023. <https://www.ibm.com/security>
- Islam, M. M., et al. (2020). The Internet of Things for Health Care: A Comprehensive Survey. *IEEE Access*, 3, 678–708.
- Kumar, A., & Patel, R. (2022). Edge Computing for Real-Time Health Monitoring. *International Journal of Cloud Applications*, 12(4), 45–58.
- Pal, K., et al. (2021). Security and Privacy Issues in IoT-based Healthcare Systems. *Procedia Computer Science*, 171, 234–243.

Ray, P. P., et al. (2021). Internet of Things for Healthcare: Progress, Challenges and Future Directions. IEEE Access, 9, 1123–1150.

Spring Framework. (2024). Spring Integration MQTT Support – Reference Documentation. <https://docs.spring.io/spring-integration/reference/mqtt.html>

Mellékletek:

A teljes program itt megtalálható:

<https://github.com/vertussgn/ChronoCare/tree/main>

Melléklettel együtt:

<https://github.com/vertussgn/ChronoCare/tree/szakdolgozat>

Szakdolgozati konzultációs lap

I. A hallgató neve, szakja: Pál László "Magyar nyelv és irodalom" informatikus (BSC)

A szakdolgozat témaválasztásának időpontja: 2025. szeptember 15.

A szakdolgozat címe:

10T alapú egyetemesi maritimus rendszer felépítése és működése
biológiai szövegek

II. A témavezető neve: Vegőce János

III. A szakdolgozat témaválasztásának megváltoztatása

A változtatás időpontja:

A szakdolgozat címe:

A témavezető neve:

IV. Konzultációk regisztrálása¹:

Sorszám	A téma felvételét követő félév száma	Időpont	Aláírás
1.	1.	2025.10.14	
2.	1.	2025.11.25	
3.	1.	2025.12.11	
4.	2.	2026.01.22	
5.	2.	2026.03.20	
6.	2.	2026.04.22	
7.	3.	—	
8.	3.	—	
9.	3.	—	
10.	3.	—	

A szakdolgozat beadható: ☒

nem adható be: ☐

Nyíregyháza, 2026. április 22.



a témavezető aláírása

¹ A hallgató félévenként 3-3 alkalommal köteles megjelenni a témavezetőnél konzultáció céljából. Nem teljesítés esetén a szakdolgozat nem adható be, illetve az adott félévben a hallgató nem kap érdemjegyet.

NYÍREGYHÁZI EGYETEM

4401 Nyíregyháza
Sóstói út 31/b

Intézményi azonosító: FI 74250

Tel: 42/599-400
Fax: 42/402-485

SZAKDOLGOZATI LAP

BA, BSc képzéshez

Név : Poll Sándor

NEPTUN-kód : LXI74R

Alapképzési szak : Magántörzsi Informatika - BSc/BA

Szakedolgozat címe: IoT alapú egészségügyi monitoring rendszer
fejlesztése krónikus betegek számára

A témavezető: Vegera József
neve

[Signature]
aláírása

A szakfelelős: Dr. Falvaskai János
neve

[Signature]
aláírása

Intézet, tanszék: Matematika és Informatika

Nyíregyháza, 20 25 év 09 hónap 22 nap

[Signature]
a hallgató aláírása

Kitöltendő 3 példányban: 1 példányt a szakfelelősnek,
1 példányt a Tanulmányi előadónak kell leadni
1 példányt a hallgatónál marad

SZAKDOLGOZATI BÍRÁLÓLAP

Név:

Intézet:

Szak: Programtervező informatikus BSc

Dolgozat címe:

A dolgozat szerkesztése stílusa:

Pont: Max: 10 pont

A szakirodalom feldolgozása, elemzése:

Pont: Max: 10 pont

A téma feldolgozási módszerének színvonala

Pont: Max: 10 pont

Az eredmények és következtetések értékelése:

Pont: Max: 15 pont

Az eredmények gyakorlati alkalmazhatósága, illetve tudományos értéke:

Pont: Max: 5 pont

Összes pontszám:

A dolgozat minősítése:

21-27 pont: elégséges (2); 28-35 pont: közepes (3); 36-42 pont: jó (4); 43-50 pont: jeles (5)

Kérdések:

Kelt,

bíráló

témavezető